UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento: Departamento de Electrónica y Telecomunicaciones

Asignatura: Fundamentos de programación

NRC: 36466

Apellidos y Nombres: Campo Quinga Rober Stheveen

Deber: Ejercicios de programación y algoritmia

Fecha: 23 de octubre de 2026

Problema 1.1. Media de dos valores

Desarrolla un programa que ingrese dos números por teclado e imprima su media.

Realiza y completa la tabla de objetos del problema 1.1:

1. Análisis del problema

Idea general: Se piden dos números y se calcula el valor que queda justo en medio de ambos: su suma dividida entre 2

Entradas: Dos números reales, a y b.

Proceso: Se suman a y b y el resultado se divide entre 2: media = (a + b) / 2.

Salida: Un mensaje con la media de a y b.

Restricción: ninguna, los números pueden ser positivos, negativos o cero.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (primer número)	a	var	real
Dato de entrada (segundo número)	b	var	real
Resultado (media)	media	var	real
Divisor de la media	2	cte	entero

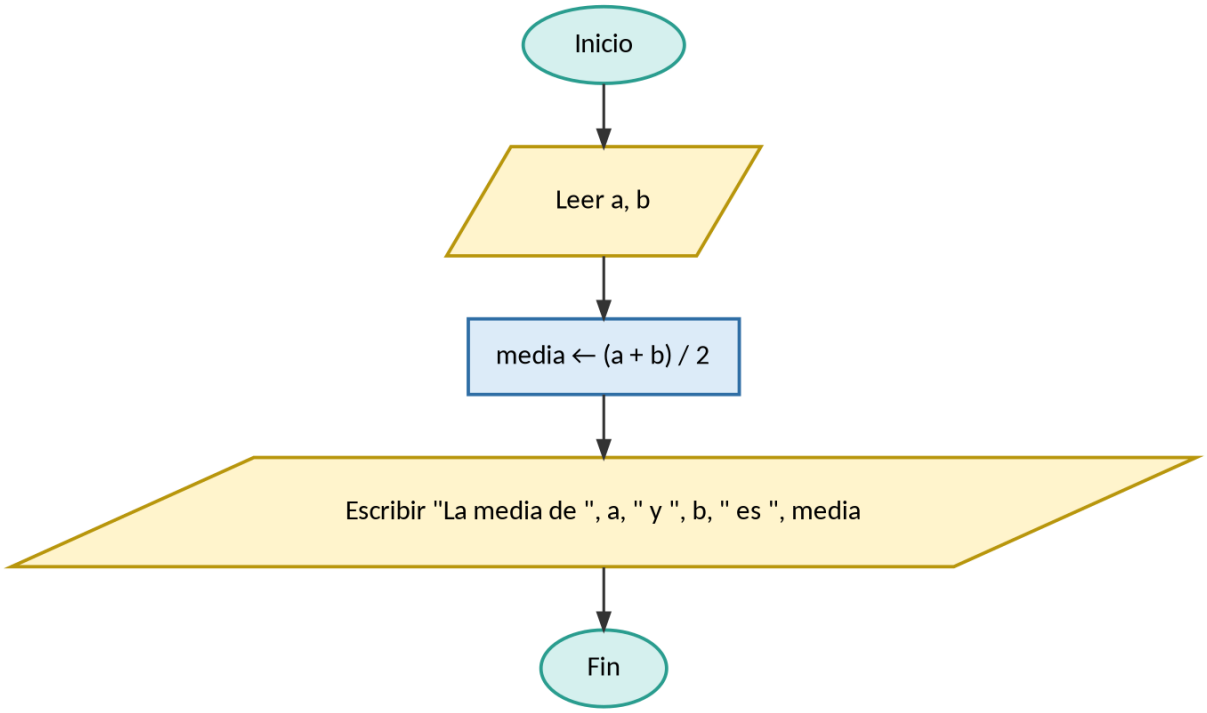
3. Seudocódigo

```
Inicio
  Leer a, b
  media ← (a + b) / 2
  Escribir "La media de ", a, " y ", b, " es ", media
Fin
```

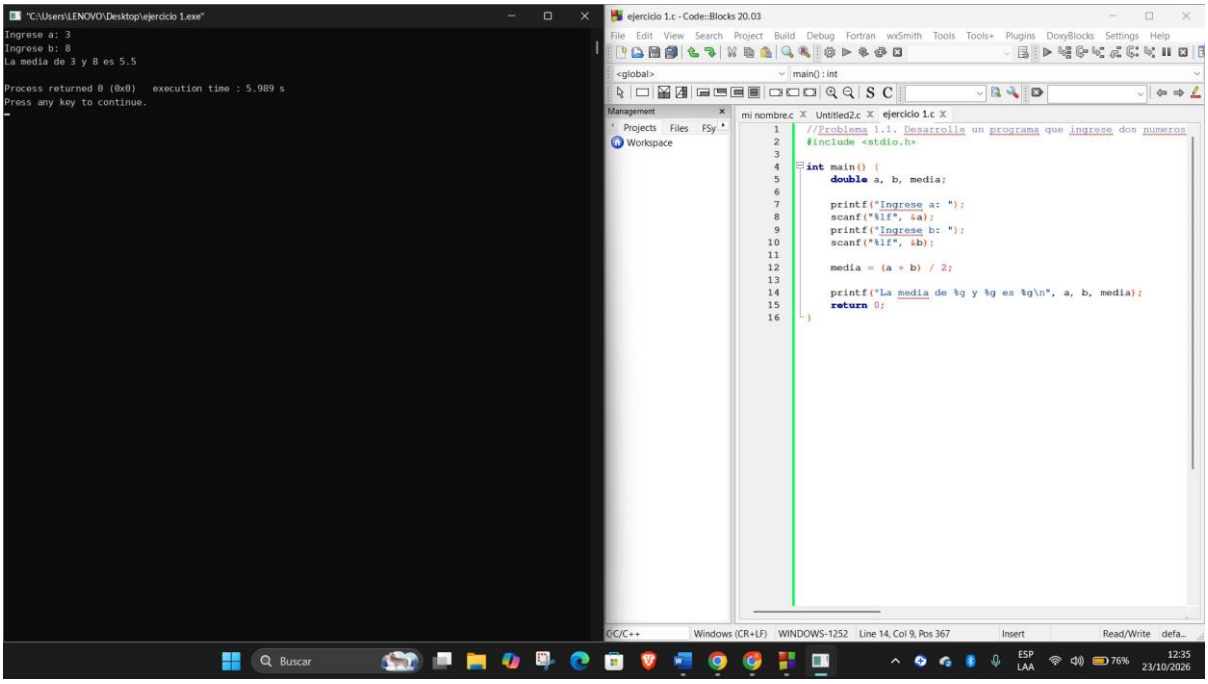
4. Prueba de escritorio

Caso	a	b	a + b	media
Normal	4	6	10	5
Normal (decimal)	3	8	11	5,5
Negativos	-5	-3	-8	-4
Opuestos	-2	2	0	0
Decimales	1,5	2,5	4	2
Límite (ceros)	0	0	0	0

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.2. Valor absoluto de x al cubo

Desarrolla un programa que lea un número por teclado y escriba por pantalla $|x|^3$.

1. Análisis del problema

Idea general: El valor absoluto le quita el signo al número, luego ese valor se eleva al cubo, por lo que el resultado nunca es negativo

Entradas: Un número real x

Proceso: si x < 0, el valor absoluto es -x; si no, es el propio x. Se multiplica ese valor tres veces por sí mismo: $|x|^3$ = absx · absx · absx.

Salida: El valor de $|x|^3$

Restricción: ninguna. Ejemplo: x = -3 → |-3| = 3 → $3^3 = 27$

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	x	var	real
Valor absoluto de x	absx	var	real
Resultado $\|x\|^3$	resultado	var	real
Cero (signo)	0	cte	entero

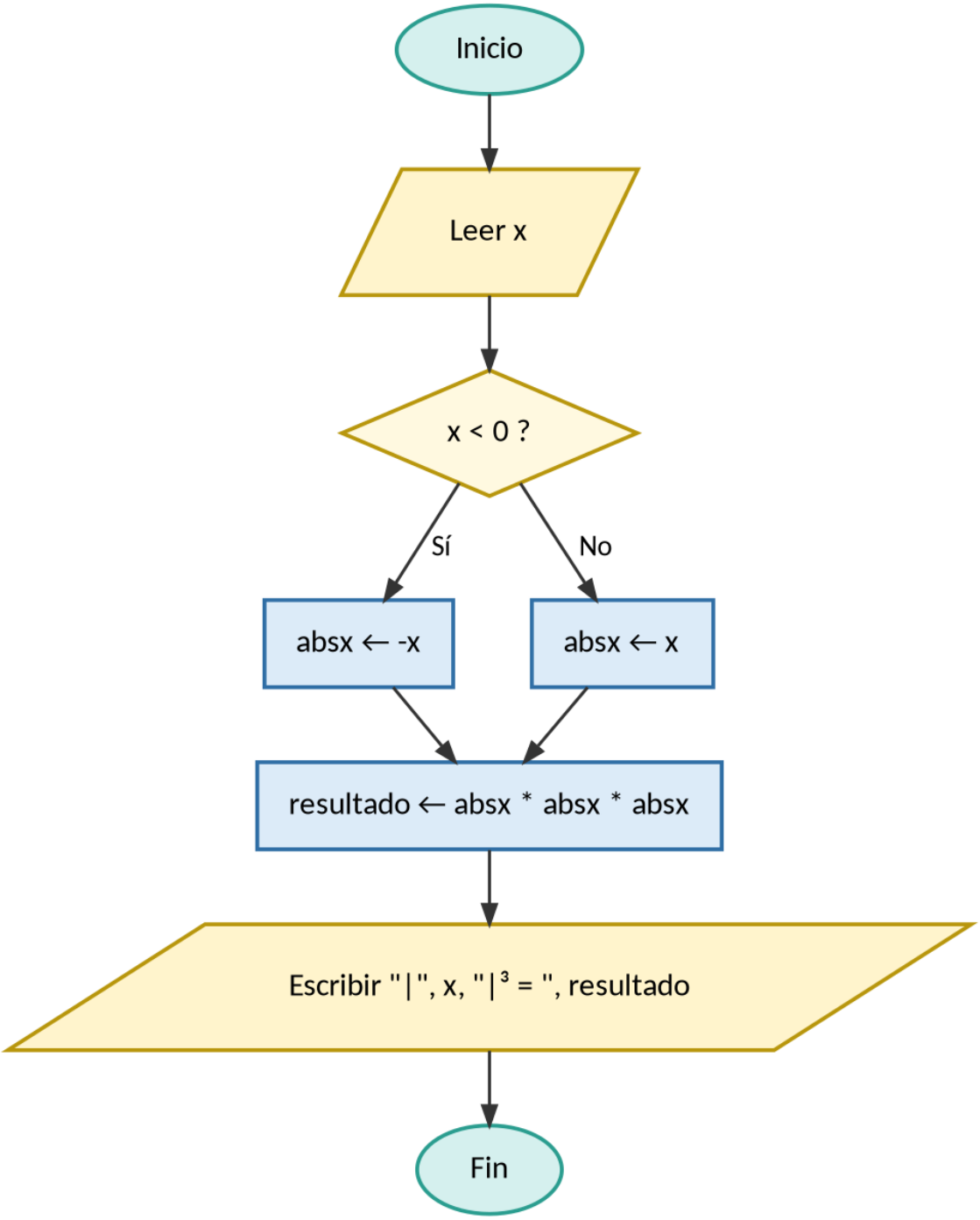
3. Seudocódigo

```
Inicio
  Leer x
  Si x < 0 entonces
    absx ← -x
  Sino
    absx ← x
  Fin Si
  resultado ← absx * absx * absx
  Escribir "|", x, "|³ = ", resultado
Fin.
```

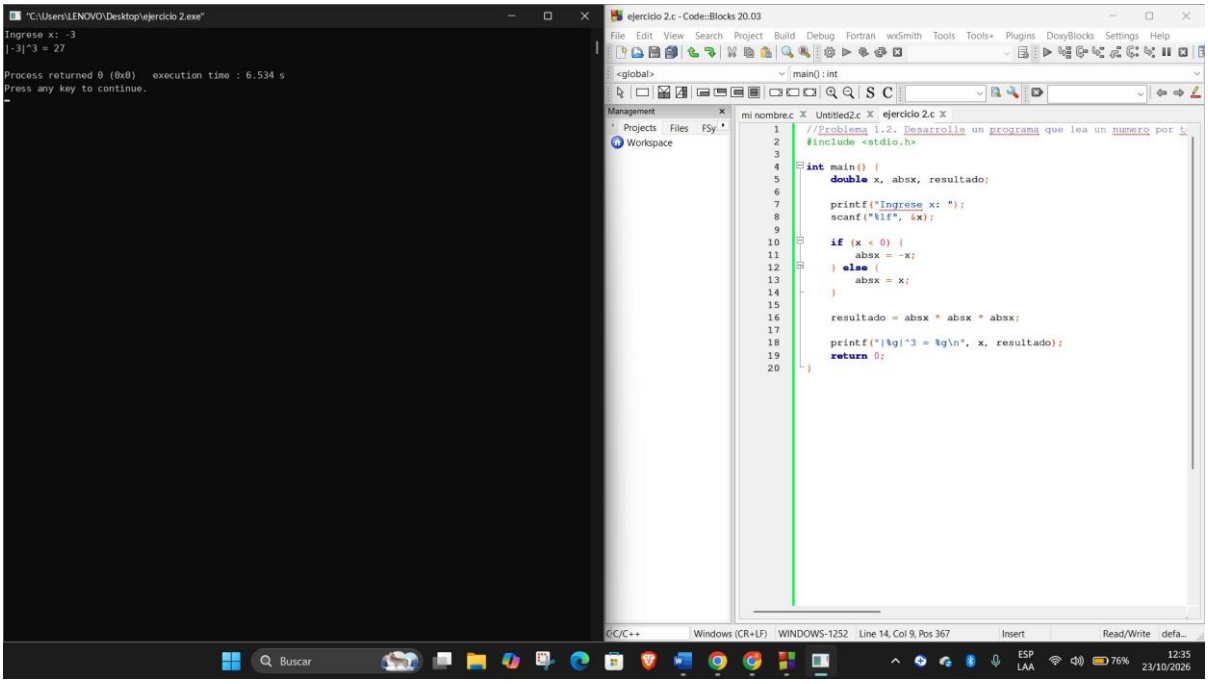
4. Prueba de escritorio

Caso	x	x < 0	absx	resultado
Normal (positivo)	2	No	2	8
Normal (negativo)	-3	Sí	3	27
Decimal	1,5	No	1,5	3,375
Decimal negativo	-0,5	Sí	0,5	0,125
Límite (cero)	0	No	0	0

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.3. Divisible

Desarrolla un programa que lea dos números enteros por teclado y determine si el primero es divisible por el segundo. Se mostrará por pantalla el resultado.

1. Análisis del problema

Entradas: Dos enteros, el dividendo x y el divisor y

Proceso: x es divisible por y cuando el resto de la división entera es cero (x % y = 0)

Salida: Un mensaje que indica si x es o no divisible por y.

Restricción: Y no puede ser 0, porque no existe la división entre cero.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (dividendo)	x	var	entero
Dato de entrada (divisor)	y	var	entero
Cero	0	cte	entero

3. Seudocódigo

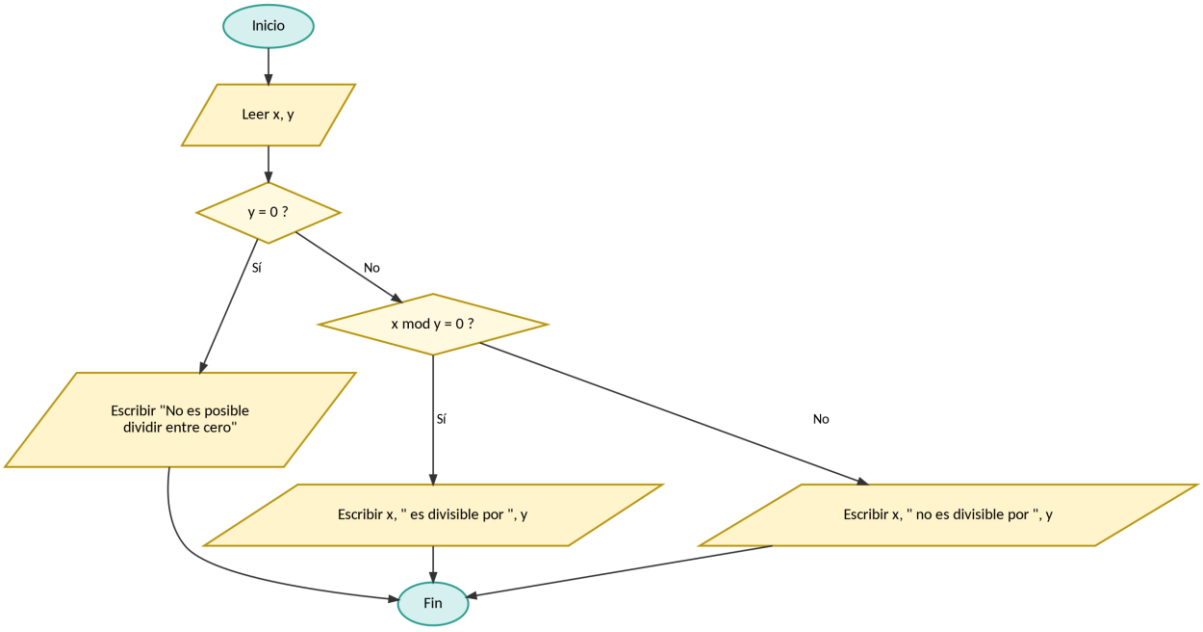
```
Inicio
  Leer x, y
  Si y = 0 entonces
    Escribir "No es posible dividir entre cero"
```

```
  Sino
    Si x mod y = 0 entonces
      Escribir x, " es divisible por ", y
      Escribir x, " no es divisible por ", y
    Fin si
Fin
```

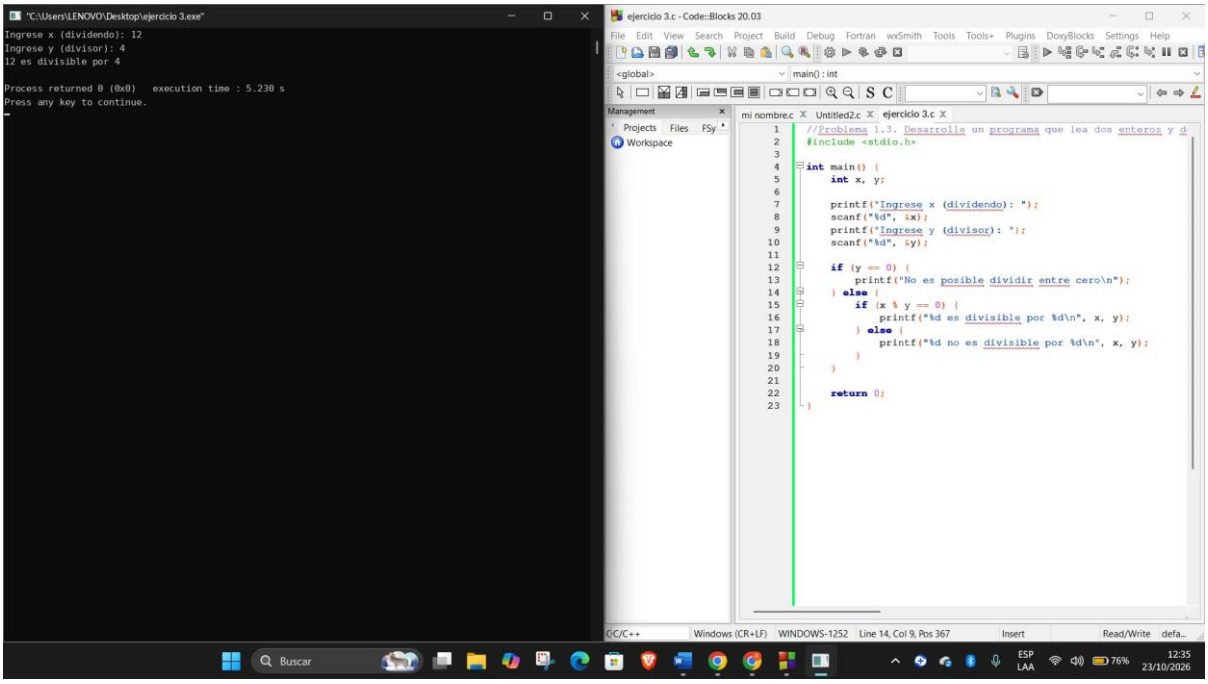
4. Prueba de escritorio

Caso	x	y	y = 0	x mod y	Salida
Normal	12	4	No	0	12 es divisible por 4
Normal	7	2	No	1	7 no es divisible por 2
Divisor igual	5	5	No	0	5 es divisible por 5
Límite (dividendo 0)	0	3	No	0	0 es divisible por 3
Negativo	-9	3	No	0	-9 es divisible por 3
Negativo	7	-2	No	1	7 no es divisible por -2
Error	8	0	Sí	-	No es posible dividir entre cero

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.4. Intervalo

Desarrolla un programa que lea un número por teclado y determine si pertenece al intervalo [0,10], indicando el resultado por pantalla.

1. Análisis del problema

Idea general: Un número está en el intervalo cerrado [0,10] si no es menor que 0 y no es mayor que 10; es decir, cuando se cumplen las dos condiciones al mismo tiempo.

Entradas: Un número real x

Proceso: Se evalúan x ≥ 0 y x ≤ 10 unidas con el operador lógico Y (&& en C). Solo si ambas son verdaderas, x pertenece al intervalo. No se escribe 0 ≤ x ≤ 10 en una sola condición porque en C se evalúa de izquierda a derecha

Salida: Un mensaje que indica si x pertenece o no al intervalo [0,10]

Restricción: Ninguna; los extremos 0 y 10 sí pertenecen porque el intervalo es cerrado.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	x	var	real
Límite inferior del intervalo	0	cte	entero
Límite superior del intervalo	10	cte	entero

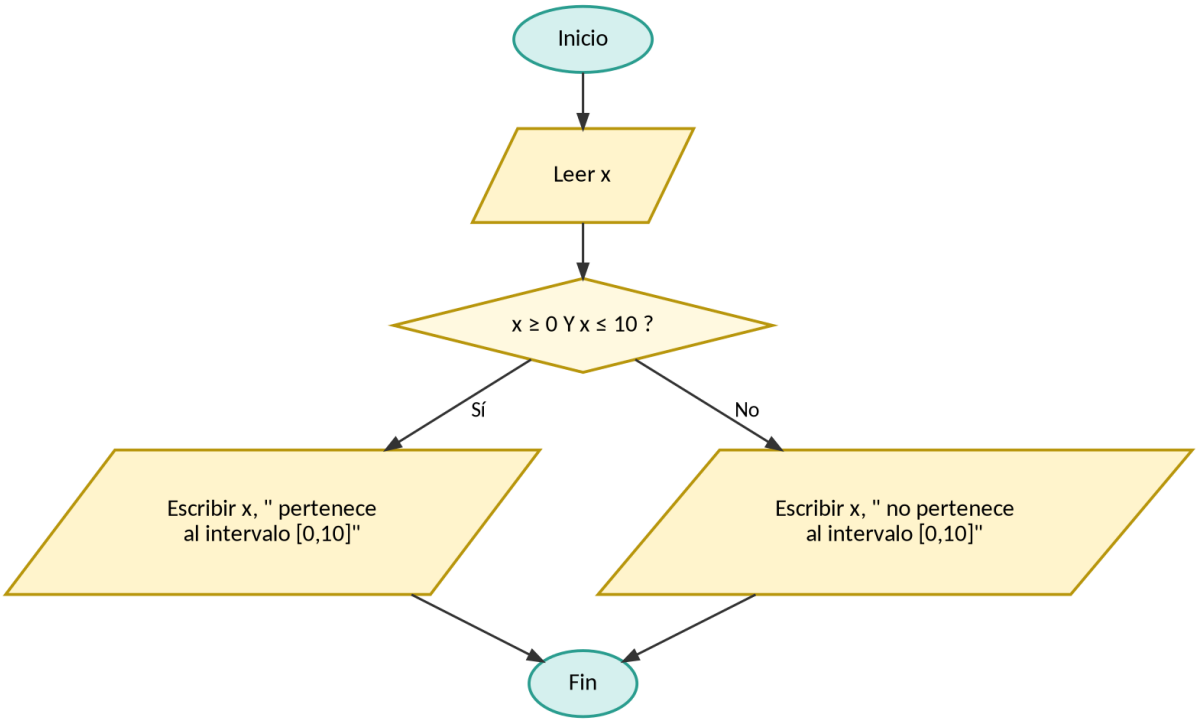
3. Seudocódigo

```
Inicio
  Leer x
  Si x >= 0 Y x <= 10 entonces
    Escribir x, " pertenece al intervalo [0,10]"
  Sino
    Escribir x, " no pertenece al intervalo [0,10]"
  Fin Si
Fin
```

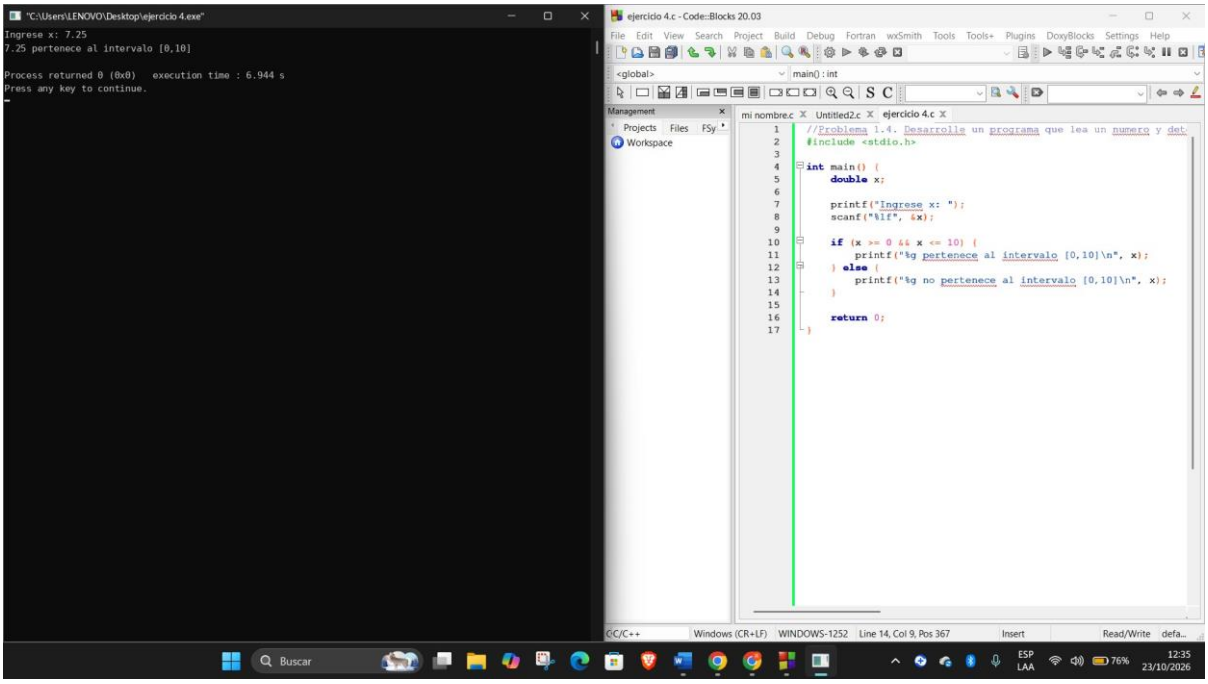
4. Prueba de escritorio

Caso	x	x ≥ 0	x ≤ 10	Y	Salida
Normal (dentro)	5	Sí	Sí	Sí	5 pertenece al intervalo [0,10]
Normal (decimal)	7,25	Sí	Sí	Sí	7,25 pertenece al intervalo [0,10]
Límite inferior	0	Sí	Sí	Sí	0 pertenece al intervalo [0,10]
Límite superior	10	Sí	Sí	Sí	10 pertenece al intervalo [0,10]
Fuera (mayor)	15	Sí	No	No	15 no pertenece al intervalo [0,10]
Fuera (menor)	-0,1	No	Sí	No	-0,1 no pertenece al intervalo [0,10]

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.5. Conversión de unidades de tiempo

Desarrolla un programa que lea por teclado un valor entero c, interpretándolo como el número de segundos que dura un evento, y calcule y muestre por pantalla cuántos días, horas, minutos y segundos representa.

1. Análisis del problema

Idea general: Se convierte una cantidad de segundos en una duración «normal»: primero se sacan los días completos, con lo que sobra sacan las horas completas, luego los minutos, y lo último que queda son los segundos.

Entradas: Un entero c, la cantidad de segundos

Proceso: días = c div 86400; resto = c mod 86400; horas = resto div 3600; resto = resto mod 3600; minutos = resto div 60; segundos = resto mod 60; Aquí 86400 = 24·60·60 y 3600 = 60·60

Salida: Un mensaje con los días, horas, minutos y segundos equivalentes.

Restricción: c no puede ser negativo (una duración no puede ser negativa)

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (segundos)	c	var	entero
Días completos	días	var	entero
Horas completas	horas	var	entero

Objeto	Nombre	Valor	Tipo
Min completos	minutos	var	entero
S sobrantes	segundos	var	entero
S que quedan por convertir	resto	var	entero
S en un día (24·60·60)	S_día = 84600	cte	entero
S en una hora (60·60)	S_hora = 3600	cte	entero
S en un minuto	S_min = 60	cte	entero
Cero (para validar)	0	cte	entero

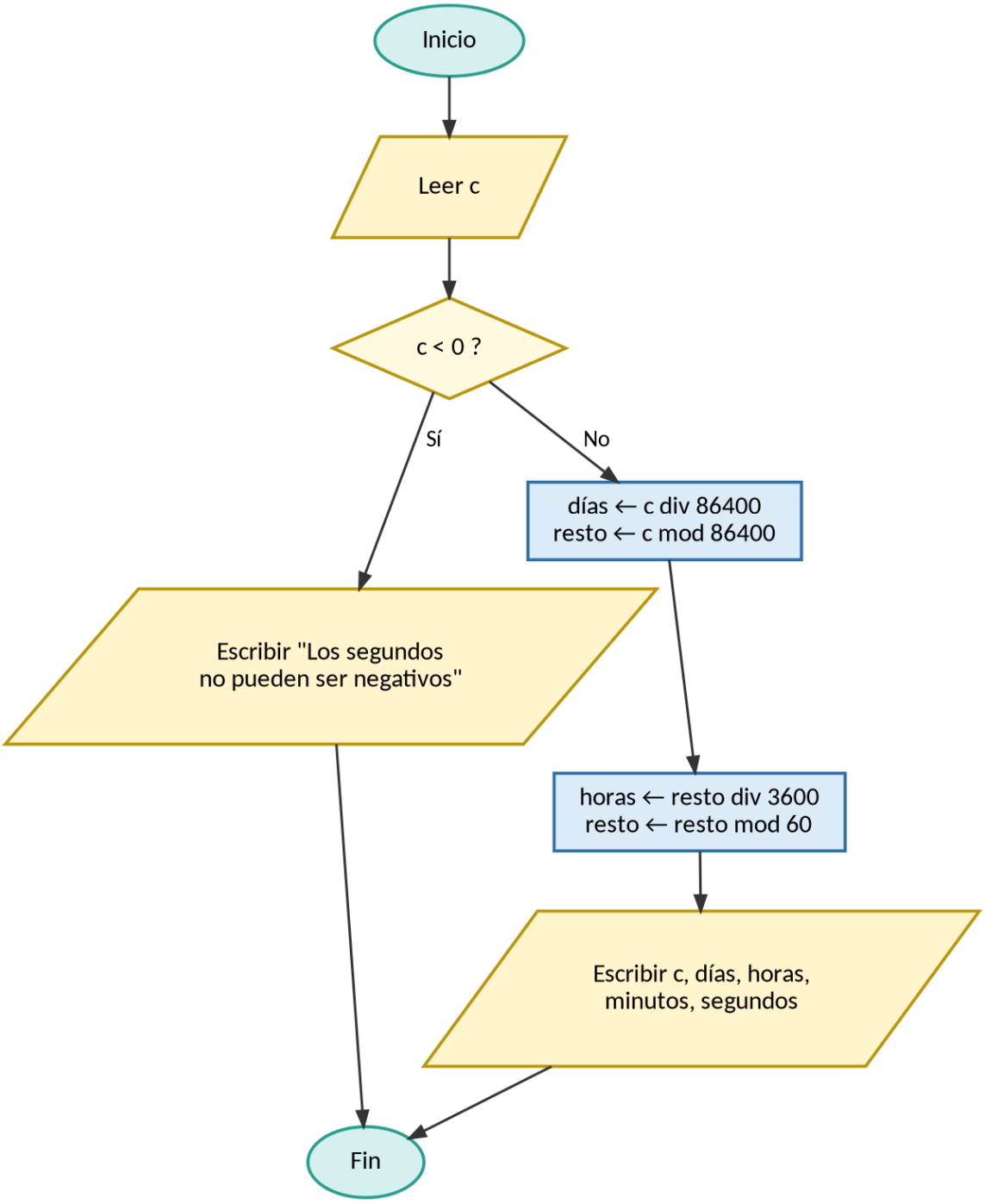
3. Seudocódigo

```
Inicio
  Leer c
  Si c < 0 entonces
    Escribir "Los segundos no pueden ser negativos"
  Sino
    días ← c div 86400
    resto ← c mod 86400
    horas ← resto div 3600
    resto ← resto mod 3600
    minutos ← resto div 60
    segundos ← resto mod 60
    Escribir c, " segundos = días ", días, ", horas ", horas, " horas ",
             minutos, " minutos y ", segundos, " segundos."
  Fin Si
Fin
```

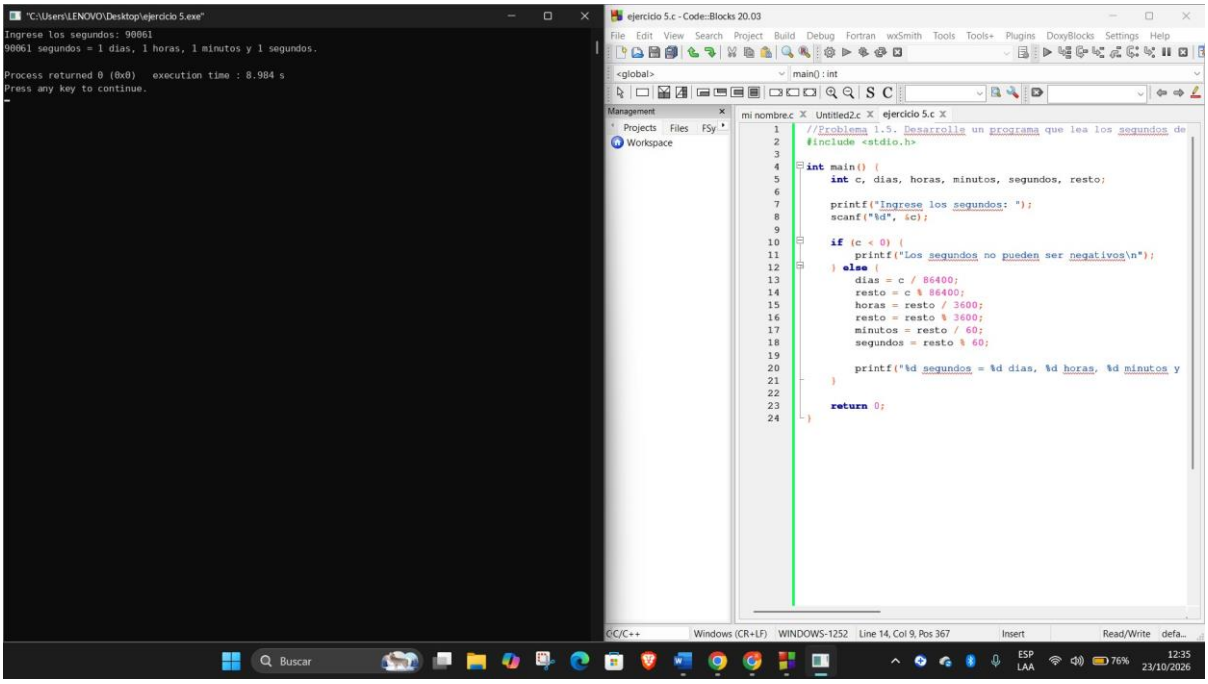
4. Prueba de escritorio

Caso	c	c < 0	días	horas	minutos	segundos
Normal	10000	No	0	2	46	40
Todos los valores	90061	No	1	1	1	1
Menos de un minuto	59	No	0	0	0	59
Exactamente 1 hora	3600	No	0	1	0	0
Exactamente 1 día	86400	No	1	0	0	0
Límite (cero)	0	No	0	0	0	0
Error	-5	Sí	-	-	-	-

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.6. Máximo, mínimo y media de alturas

Desarrolla un programa que lea por teclado un número indeterminado de alturas y deje de leer cuando se introduzca una negativa. El programa debe calcular e imprimir la media, el máximo y el mínimo de las alturas introducidas.

1. Análisis del problema

Idea general: Se leen alturas una por una sin saber cuántas serán; el ingreso de un número negativo es la señal para parar (centinela). Mientras se leen, se lleva la suma, la cuenta, la mayor y la menor altura, sin guardar todas

Entradas: Alturas h (reales), hasta que se ingrese una negativa

Proceso: Por cada altura válida (h ≥ 0): se suma a suma, se aumenta cont y se actualizan máximo y mínimo (la primera altura inicializa ambos).

Salida: La media, el máximo y el mínimo de las alturas.

Restricción: La altura negativa solo termina la lectura; no cuenta como dato. Si la primera altura ya es negativa no hay datos y se informa, lo que evita dividir entre cero

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de altura (entrada)	h	var	real
Suma de las alturas	suma	var	real

Objeto	Nombre	Valor	Tipo
Cantidad de alturas leídas	cont	var	entero
Altura máxima	maximo	var	real
Altura mínima	minimo	var	real
Media de las alturas	media	var	real
Cero	0	cte	entero

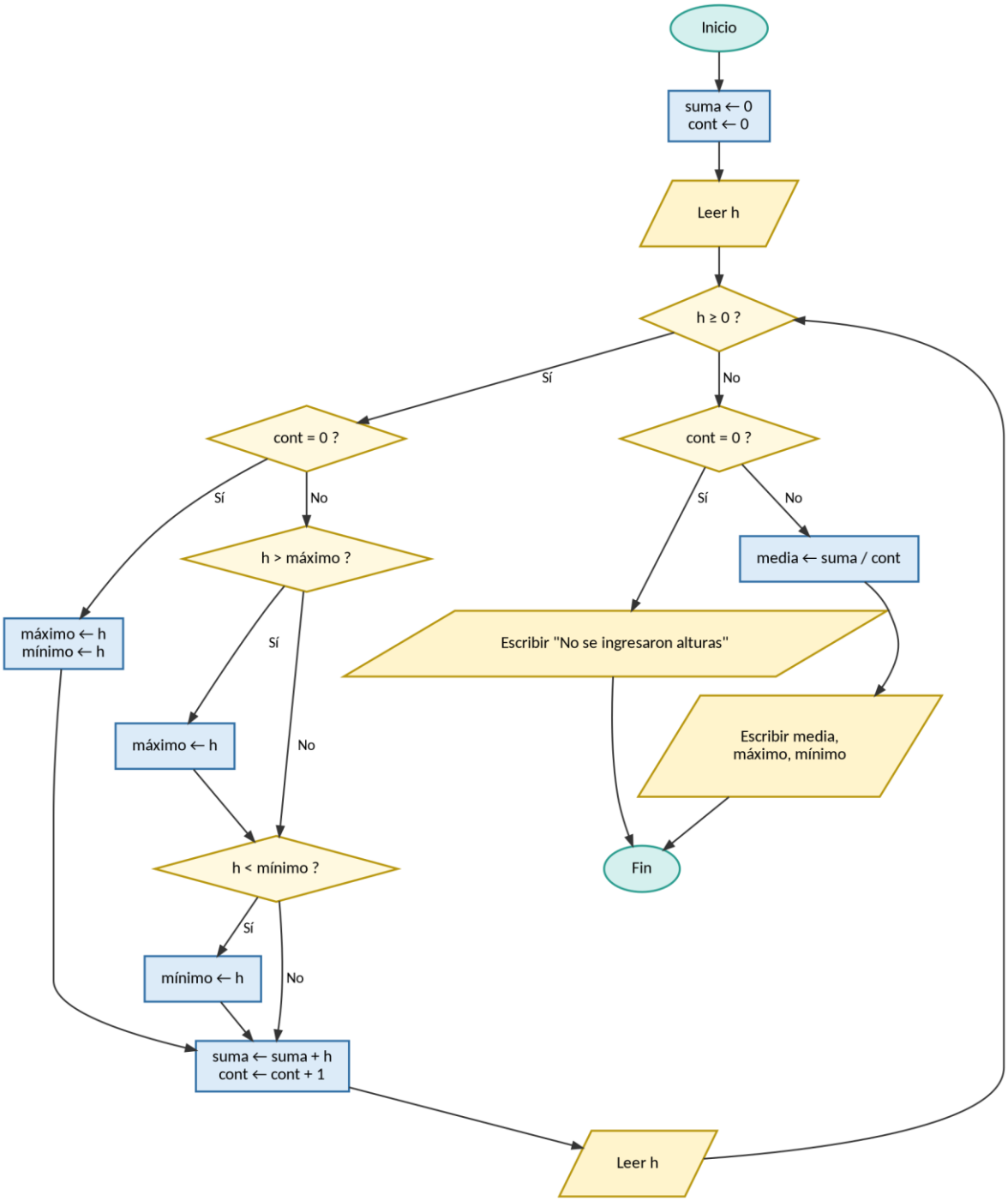
3. Seudocódigo

```
Inicio
  suma ← 0
  cont ← 0
  Leer h
  Mientras h >= 0 hacer
    Si cont = 0 entonces
      maximo ← h
      minimo ← h
    Sino
      Si h > maximo entonces
        maximo ← h
      Fin Si
      Si h < minimo entonces
        minimo ← h
      Fin Si
    Fin Si
    suma ← suma + h
    cont ← cont + 1
    Leer h
  Fin Mientras
  Si cont = 0 entonces
    Escribir "No se ingresaron alturas"
  Sino
    media ← suma / cont
    Escribir "Media: ", media
    Escribir "Mínimo: ", minimo
    Escribir "Máximo: ", maximo
  Fin Si
Fin
```

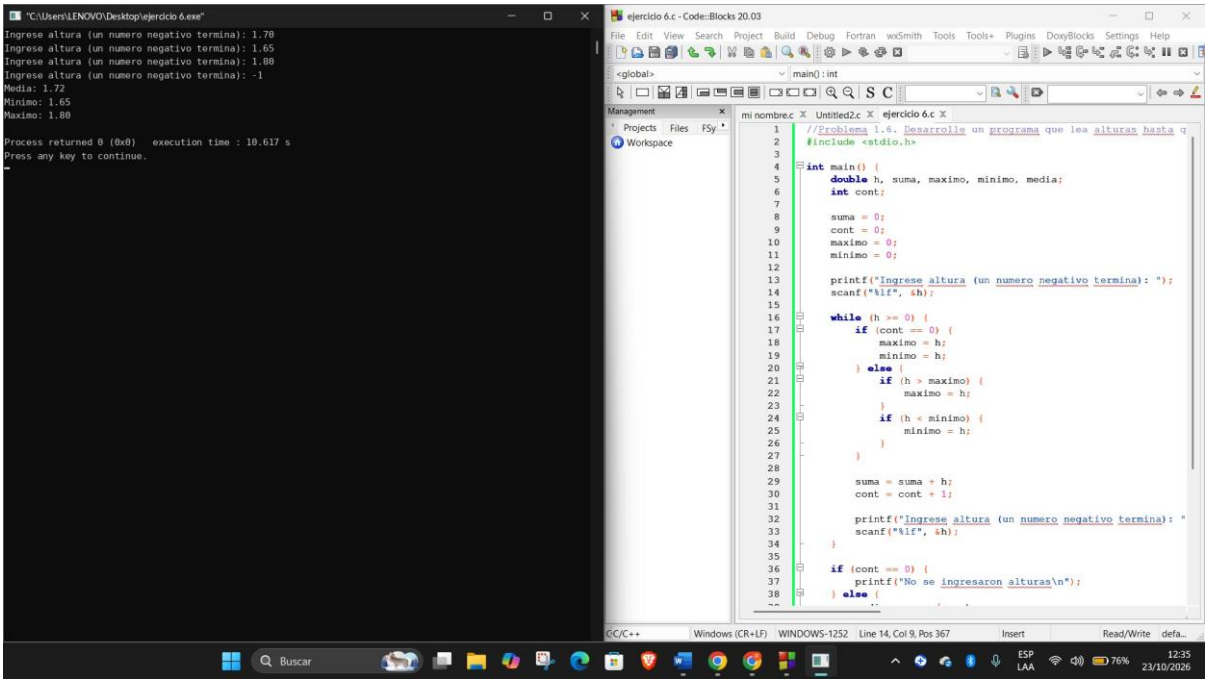
4. Prueba de escritorio

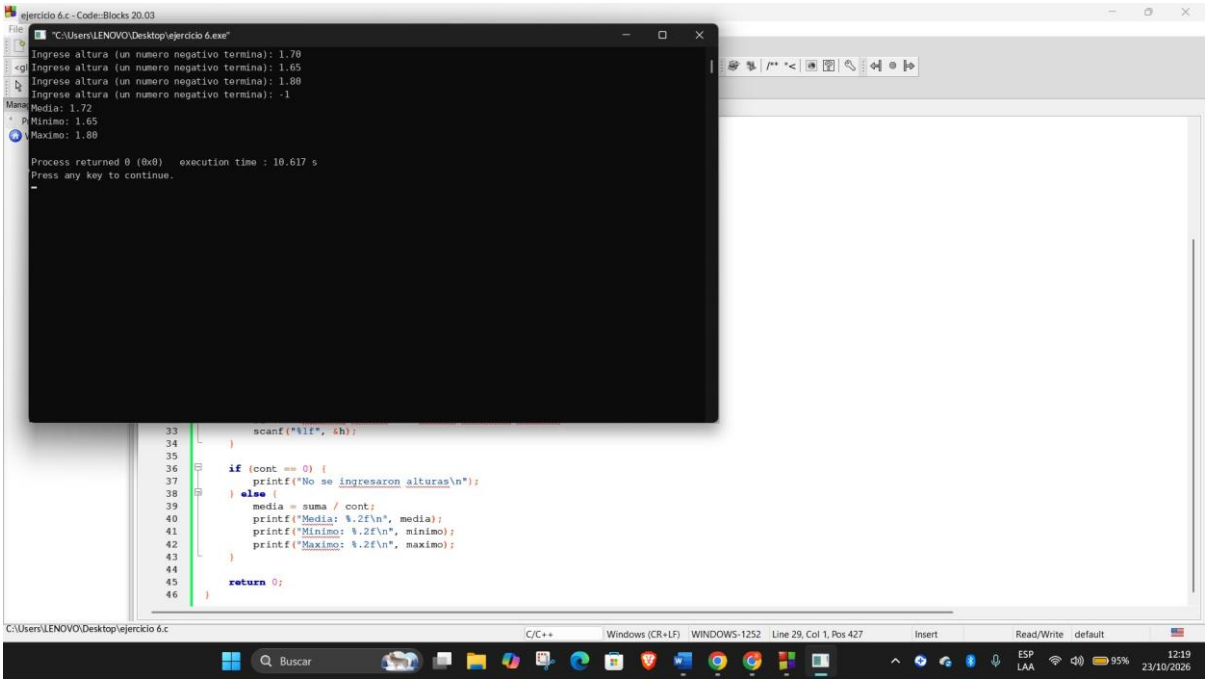
Caso	Alturas ingresadas	cont	Media	Máximo	Mínimo
Normal	1,70; 1,65; 1,80; -1	3	1,72	1,80	1,65
Normal	1,60; 1,90; 1,75; 1,55; -2	4	1,70	1,90	1,55
Límite (una sola altura)	1,50; -1	1	1,50	1,50	1,50
Error (sin datos)	-1	0	No se ingresó	-	-

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)





Problema 1.7. Raíz cuadrada entera

Desarrolla un programa que calcule la raíz cuadrada entera por defecto de un número N positivo dado, que se define como el mayor número entero R > 0 tal que $R^2 \leq N$

1. Análisis del problema

Idea general: La raíz cuadrada entera por defecto de N es el entero más grande cuyo cuadrado no supera a N. Se prueba i = 1, 2, 3, ... hasta que i^2 se pasa de N; el último valor que no se pasaba es la respuesta.

Entradas: Un número entero positivo N

Proceso: Se empieza con i = 1; mientras $i^2 \leq N$, se suma 1 a i; cuando $i^2 > N$ el ciclo termina; como i ya se pasó de la raíz, R = i − 1

Salida: Un mensaje con la raíz cuadrada entera R

Restricción: N debe ser mayor que 0, se valida antes de iniciar el ciclo. Ejemplo: N = 10 → i = 1, 2, 3 cumplen; i = 4 no (16 > 10) → R = 3

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (número)	N	var	entero
Contador de prueba	i	var	entero largo
Raíz entera por defecto	R	var	entero largo
Valor inicial del contador	1	cte	entero
Cero (para validar)	0	cte	entero

3. Seudocódigo

```
Inicio
  Leer N
  Si N <= 0 entonces
    Escribir "N debe ser positivo"
  Sino
    i ← 1
    Mientras i * i <= N hacer
      i ← i + 1
    Fin mientras
    R ← i - 1
    Escribir "La raíz cuadrada entera de ", N, " es ", R
  Fin Si
Fin
```

4. Prueba de escritorio

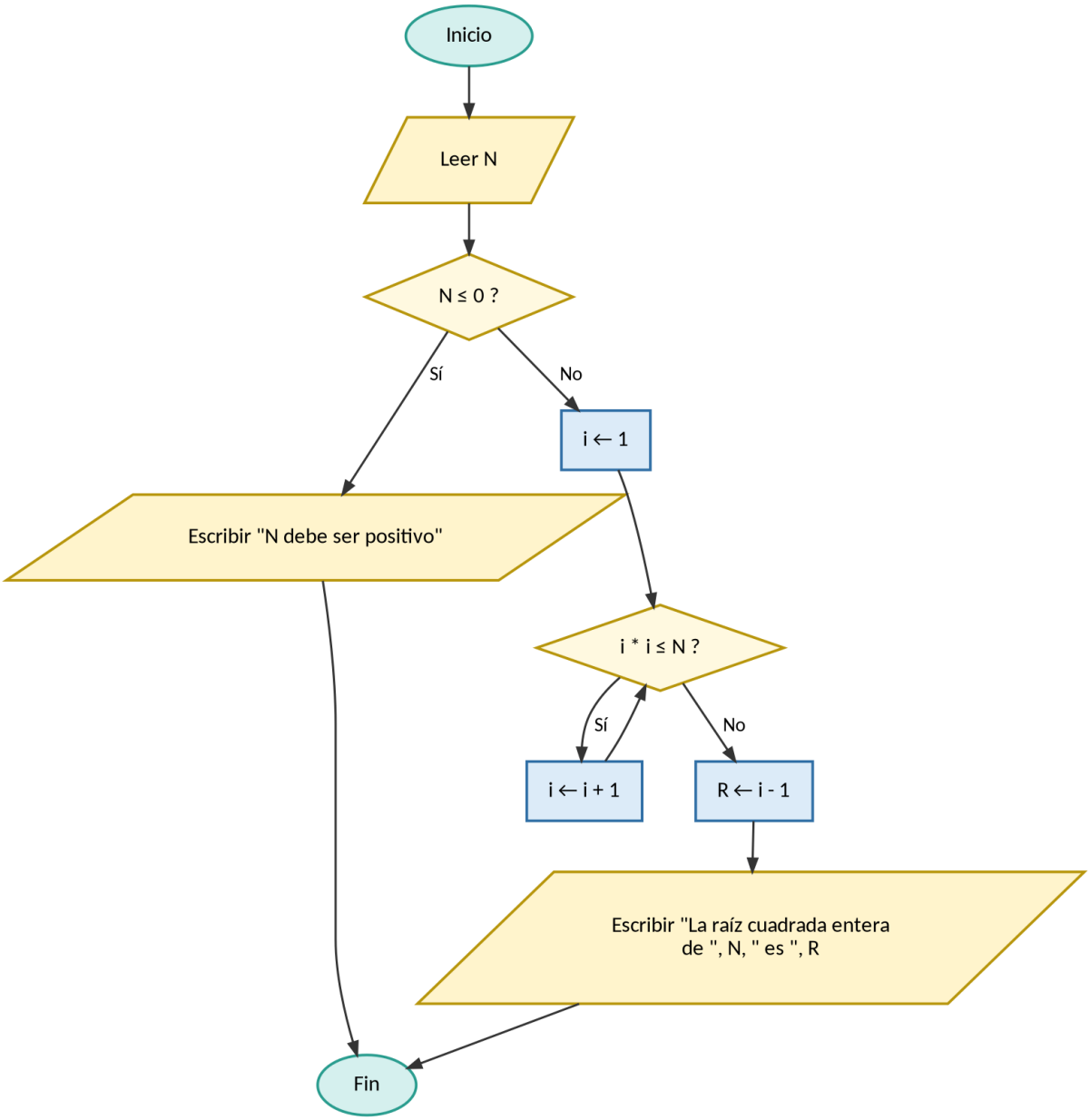
Caso	N	N ≤ 0	i final ($i^2 > N$)	R = i − 1	Salida
Normal	10	No	4	3	La raíz cuadrada entera de 10 es 3
Normal	26	No	6	5	La raíz cuadrada entera de 26 es 5
Cuadrado perfecto	16	No	5	4	La raíz cuadrada entera de 16 es 4
Límite	1	No	2	1	La raíz cuadrada entera de 1 es 1
Límite (no cuadrado)	2	No	2	1	La raíz cuadrada entera de 2 es 1
Error	0	Sí	-	-	N debe ser positivo

Caso	N	N ≤ 0	i final ($i^2 > N$)	R = i − 1	Salida
Error	-4	Sí	-	-	N debe ser positivo

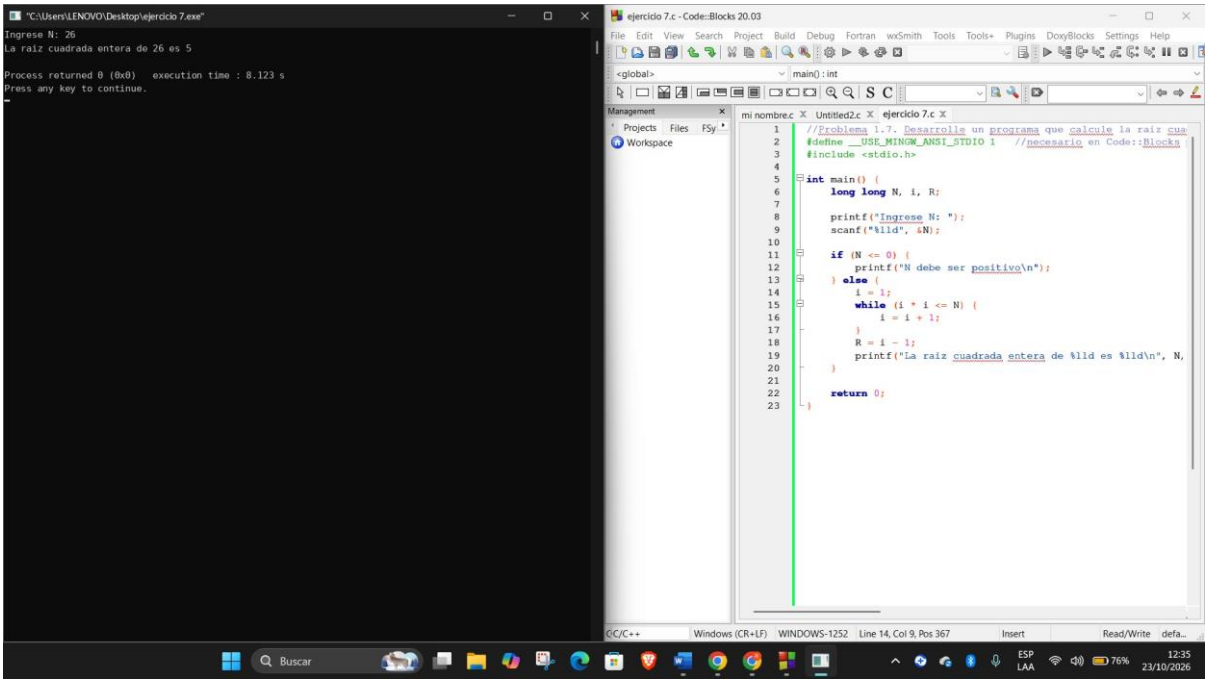
Traza del caso N = 10

i	i * i	i * i ≤ N	Acción
1	1	Sí	i ← 2
2	4	Sí	i ← 3
3	9	Sí	i ← 4
4	16	No	Fin del ciclo: R = 4 − 1 = 3

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.8. Número primo

Desarrolla un programa que ingrese por teclado un número entero mayor que 1 y determine si es primo o no.

1. Análisis del problema

Idea general: Un número es primo si solo es divisible entre 1 y entre sí mismo. Se intenta dividirlo entre 2, 3, 4, ...; si alguno lo divide exactamente, ya no es primo.

Entradas: Un entero n > 1

Proceso: se prueban los divisores d = 2, 3, ... mientras d·d ≤ n (hasta llegar hasta √n: si n tuviera un divisor mayor que √n, tendría también uno menor que √n que ya se habría probado). Si algún d cumple n mod d = 0, n no es primo

Salida: Un mensaje que indica si n es primo o no

Restricción: n > 1; el 1, el 0 y los negativos no son primos y se rechazan con un mensaje.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Divisor de prueba	d	var	entero
Indicador de primalidad	esPrimo	var	lógico

Objeto	Nombre	Valor	Tipo
Primer divisor a probar	2	cte	entero
Límite de validación	1	cte	entero

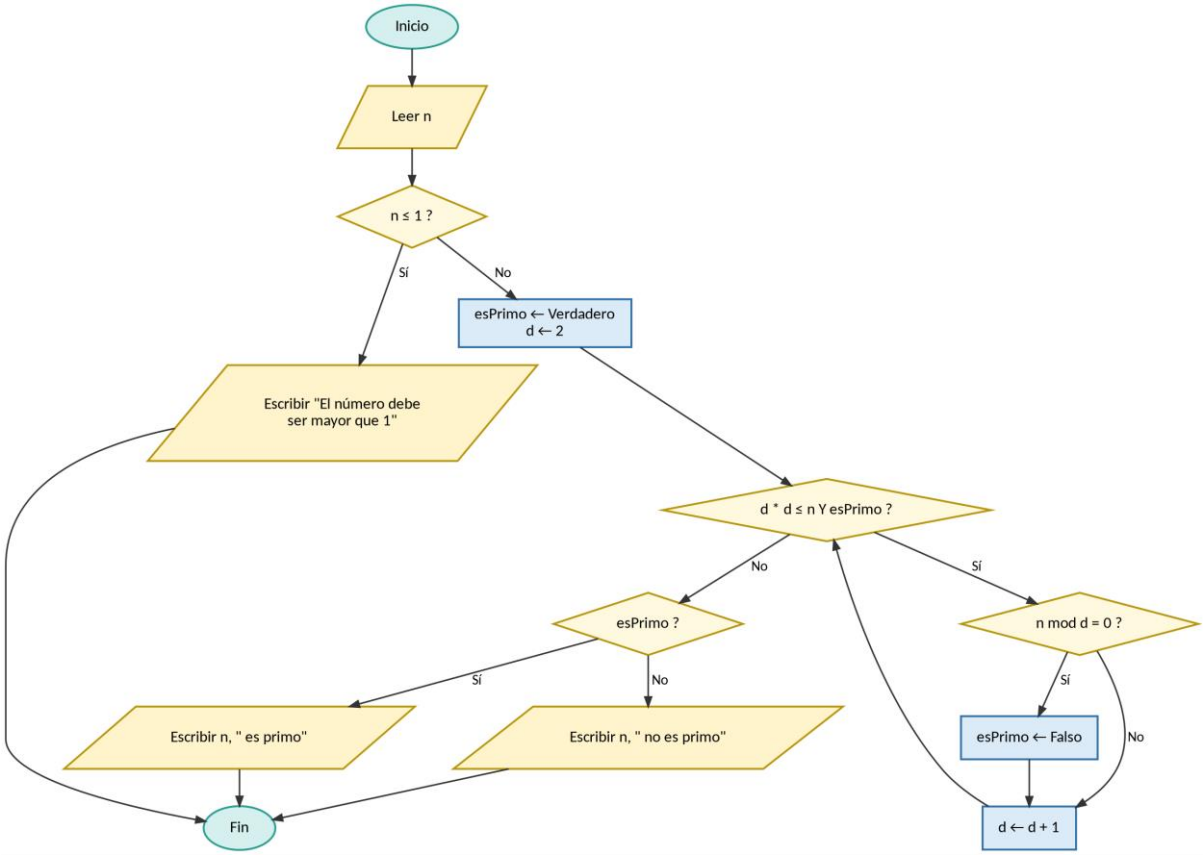
3. Seudocódigo

```
Inicio
  Leer n
  Si n <= 1 entonces
    Escribir "El número debe ser mayor que 1"
  Sino
    esPrimo ← Verdadero
    d ← 2
    Mientras d * d <= n Y esPrimo hacer
      Si n mod d = 0 entonces
        esPrimo ← Falso
      Fin Si
      d ← d + 1
    Fin Mientras
    Si esPrimo entonces
      Escribir n, " es primo"
    Sino
      Escribir n, " no es primo"
    Fin Si
  Fin Si
Fin
```

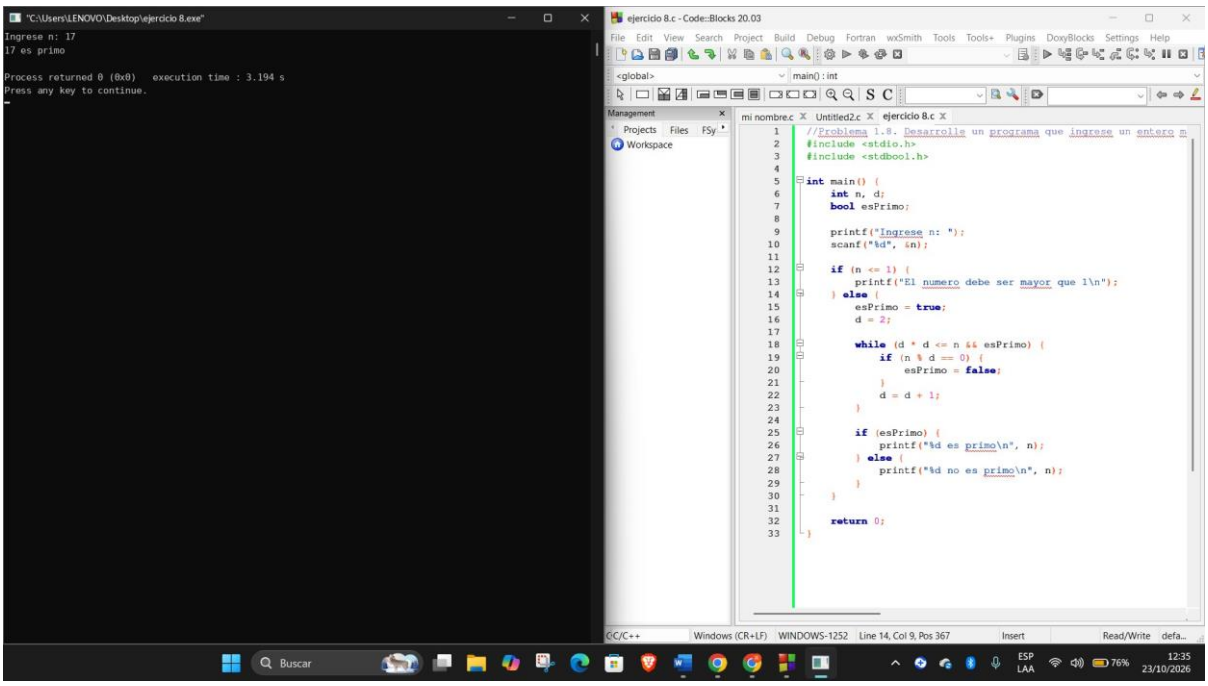
4. Prueba de escritorio

Caso	n	n ≤ 1	Divisores probados (d)	esPrimo	Salida
Normal (primo)	17	No	2, 3, 4	Verdadero	17 es primo
Normal (compuesto)	9	No	2, 3	Falso	9 no es primo
Compuesto par	10	No	2	Falso	10 no es primo
Cuadrado de un primo	25	No	2, 3, 4, 5	Falso	25 no es primo
Límite (menor primo)	2	No	ninguno (2*2 > 2)	Verdadero	2 es primo
Límite	3	No	ninguno (2*2 > 3)	Verdadero	3 es primo
Error	1	Sí	-	-	Número > 1
Error	-7	Sí	-	-	Número > 1

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.9. Números primos comprendidos entre dos números dados

Desarrolla un programa que muestre por pantalla todos los números primos comprendidos entre dos números enteros introducidos por teclado.

1. Análisis del problema

Idea general: Es el problema 1.8 repetido para cada número del rango: se recorre cada n desde el menor hasta el mayor y se muestran los que resulten primos.

Entradas: dos enteros a y b (en cualquier orden)

Proceso: si a > b se intercambian; si a < 2 se sube a 2, porque no hay primos menores; para cada n de a hasta b se aplica la prueba de primalidad del 1.8; si n es primo se muestra y se cuenta.

Salida: La lista de primos del rango (incluidos los extremos) o un mensaje si no hay ninguno

Restricción: ninguna sobre el orden o el signo de los datos; el programa los corrige.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (primer extremo)	a	var	Entero
Dato de entrada (segundo extremo)	b	var	Entero
Auxiliar para el intercambio	aux	var	Entero
Número que se está evaluando	n	var	Entero
Divisor de prueba	d	var	Entero
Indicador de primalidad	esPrimo	var	lógico
Cantidad de primos encontrados	cont	var	Entero
Menor número primo	2	cte	Entero

3. Seudocódigo

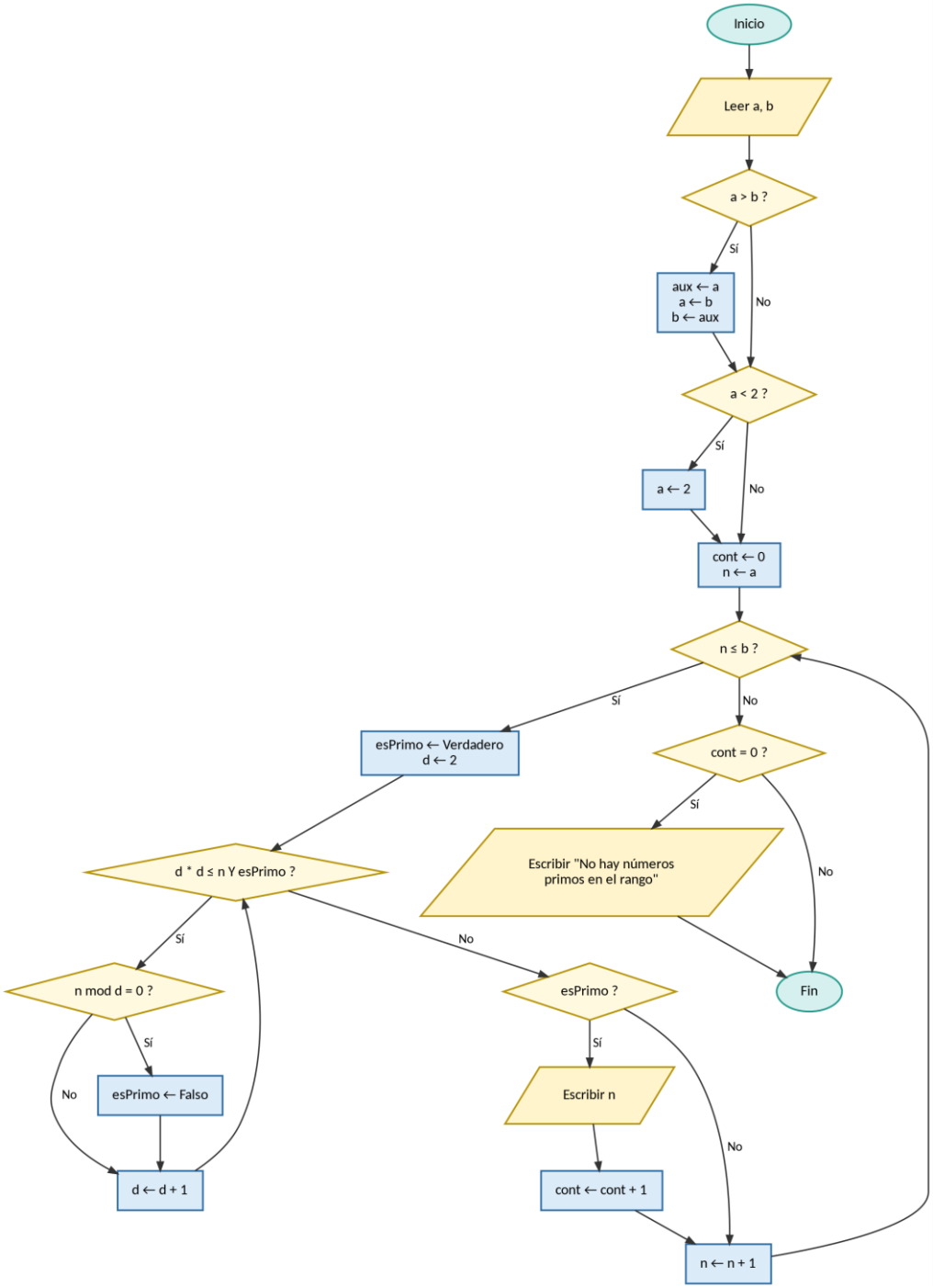
```
Inicio
  Leer a, b
  Si a > b entonces
    aux ← a
    a ← b
    b ← aux
  Fin Si
  Si a < 2 entonces
    a ← 2
  Fin Si
  Cont ← 0
  n ← a
  Mientras n <= b hacer
    esPrimo ← Verdadero
    d ← 2
    Mientras d * d <= n Y esPrimo hacer
      Si n mod d = 0 entonces
        esPrimo ← Falso
      Fin Si
```

```
      d ← d + 1
    Fin Mientras
    Si esPrimo entonces
      Escribir n
      cont ← cont + 1
  Fin Mientras
  Si cont = 0 entonces
    Escribir "No hay números primos en el rango"
  Fin Si
Fin
```

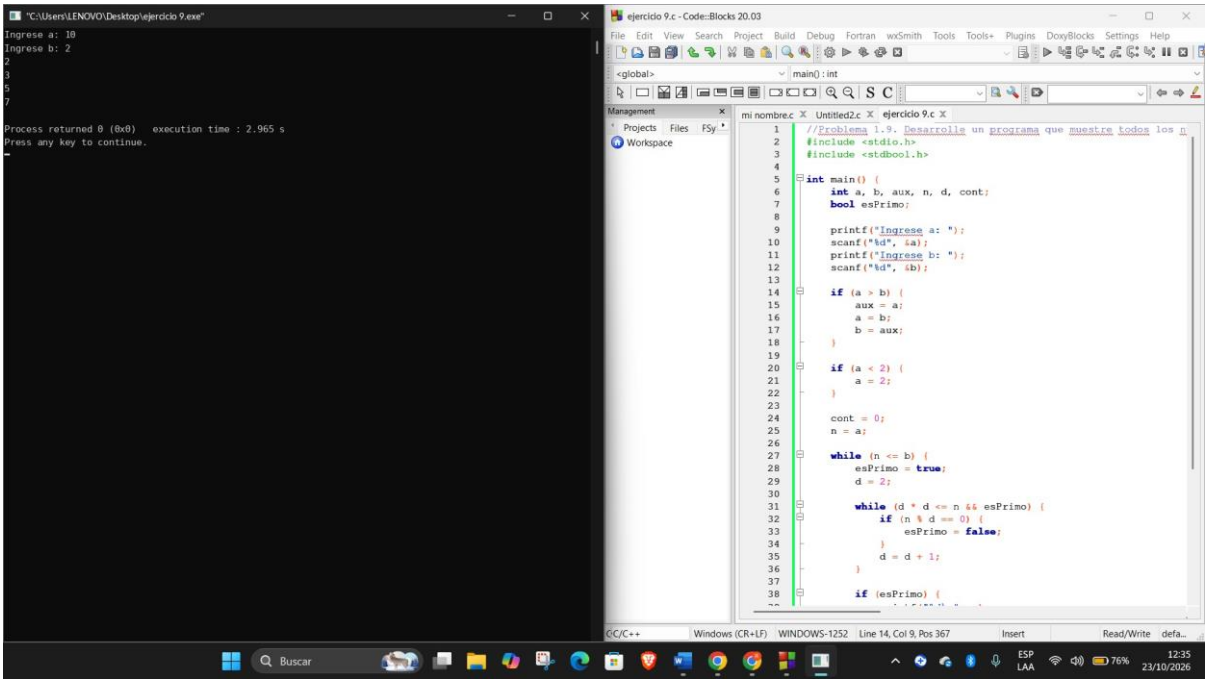
4. Prueba de escritorio

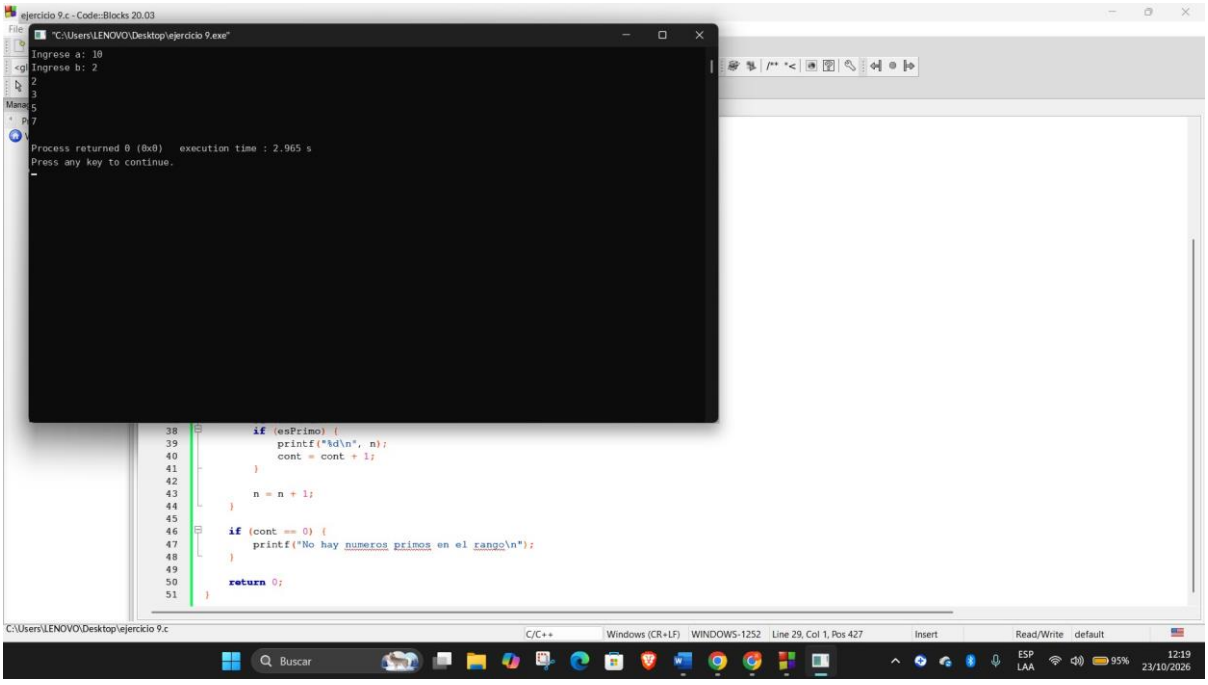
Caso	a	b	Rango revisado	Salida
Normal	10	30	10 a 30	11, 13, 17, 19, 23, 29
Datos invertidos	30	10	10 a 30 (se intercambian)	11, 13, 17, 19, 23, 29
Desde 1	1	10	2 a 10 (a se sube a 2)	2 3 5 7
Un solo valor	7	7	7 a 7	7
Con negativos	-5	3	2 a 3	2 3
Sin primos	14	16	14 a 16	No hay en el rango
Límite	0	1	2 a 1 (rango vacío)	No hay en el rango

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)





Problema 1.10. Factorial

Desarrolla un programa que lea un número entero n ≥ 0 y calcule n! = 1·2·…·n, sabiendo que 0! = 1

1. Análisis del problema

Idea general: n! es el producto de todos los enteros de 1 hasta n. Se parte de un acumulador en 1 y, en cada vuelta, se multiplica por el siguiente entero.

Entrada: Un entero n

Proceso: fact ← 1; para i = 1, 2, ..., n: fact ← fact * i. Si n = 0 el ciclo no se ejecuta y queda 0! = 1, que es lo correcto

Salida: El valor de n!

Restricción: n ≥ 0 (el factorial de un negativo no existe) y n ≤ 20 (20! es el mayor factorial que cabe en un entero de 64 bits)

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Acumulador del producto	fact	var	entero largo
Contador del ciclo	i	var	entero
Máximo n permitido	20	cte	entero
Cero (para validar)	0	cte	entero

3. Seudocódigo

```
Inicio
  Leer n
  Si n < 0 entonces
    Escribir "El factorial no está definido para negativos"
  Sino
    Si n > 20 entonces
      Escribir "n es demasiado grande (máximo 20)"
    Sino
      fact ← 1
      i ← 1
      Mientras i <= n hacer
        fact ← fact * i
        i ← i + 1
      Fin mientras
      Escribir n, "! = ", fact
Fin
```

4. Prueba de escritorio

Caso	n	n < 0	n > 20	fact	Salida
Normal	5	No	No	120	5! = 120
Normal	10	No	No	3628800	10! = 3628800
Límite (n = 0)	0	No	No	1	0! = 1
Límite (n = 1)	1	No	No	1	1! = 1
Límite (máximo)	20	No	No	2432902008176640000	20! = 2432902008176640000

Caso	n	n < 0	n > 20	fact	Salida
Error	-3	Sí	-	-	Factorial no negativo
Error	21	No	Sí	-	n demasiado grande (max 20)

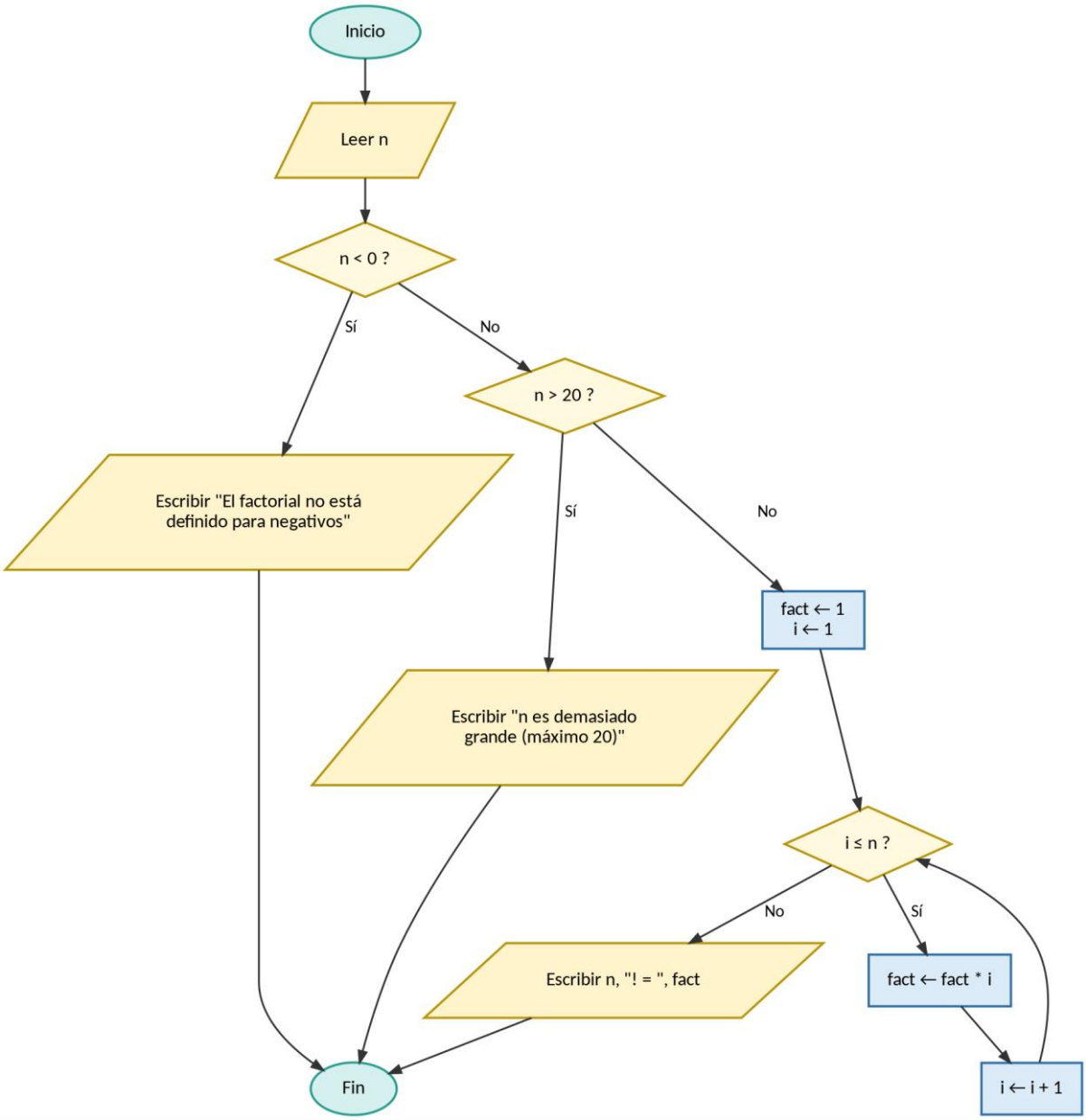
Para pensar: ¿Qué ocurre si el usuario ingresa un número negativo?

El factorial no está definido para los negativos. Sin validación, la condición i ≤ n sería falsa desde el inicio, el ciclo no se ejecutaría y el programa mostraría 1, un resultado incorrecto sin ningún aviso. Por eso se valida n < 0 y se muestra un mensaje de error. Ejemplo: n = -3 → Factorial no definido para negativos.

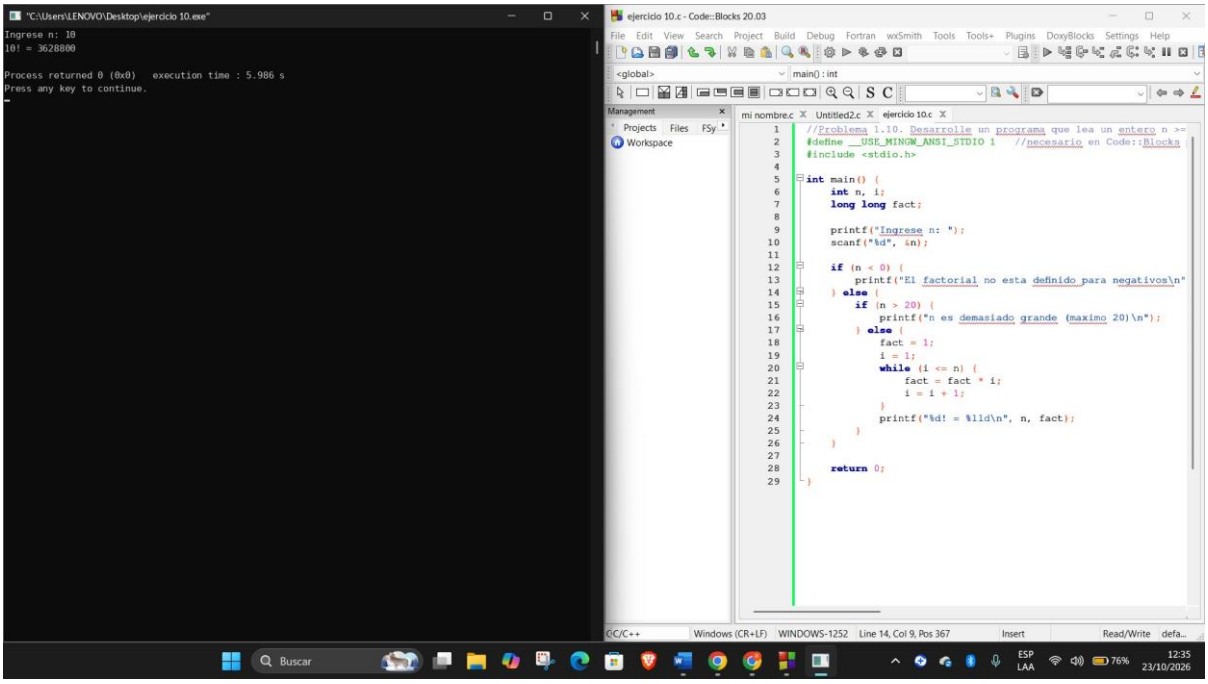
Para pensar: ¿Y con n = 20 en una variable de tipo int?

Un int de 32 bits solo llega hasta 2 147 483 647. Como 12! = 479 001 600 todavía cabe, pero 13! = 6 227 020 800 ya no; con n = 20 el resultado se desborda y sale un número equivocado.

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.11. Serie de Fibonacci

Desarrolla un programa que lea un entero n ≥ 1 y muestre los primeros n términos de la serie de Fibonacci (0, 1, 1, 2, 3, 5, 8, ...), donde cada término es la suma de los dos anteriores.

Ejemplo n = 7 → 0 1 1 2 3 5 8.

1. Análisis del problema

Idea general: Cada término es la suma de los dos anteriores, empezando con 0 y 1. Solo se necesita recordar los dos últimos términos e ir avanzando.

Entradas: Un entero n, la cantidad de términos a mostrar

Proceso: ant = 0 y act = 1. En cada una de las n vueltas: se muestra ant, se calcula sig = ant + act y se avanza (ant ← act, act ← sig).

Salida: los primeros n términos de la serie, separados por espacios

Restricción: 1 ≤ n ≤ 90 (hasta ahí los valores caben en un entero de 64 bits)

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (cantidad de términos)	n	var	entero
Término anterior	ant	var	entero largo
Término actual	act	var	entero largo

Objeto	Nombre	Valor	Tipo
Término siguiente	sig	var	entero largo
Contador del ciclo	i	var	entero
Máximo n permitido	90	cte	entero

3. Seudocódigo

```
Inicio
  Leer n
  Si n < 1 o n > 90 entonces
    Escribir "Ingrese un valor entre 1 y 90"
  Sino
    ant ← 0
    act ← 1
    Para i ← 1 hasta n hacer
      Escribir ant
      sig ← ant + act
      ant ← act
      act ← sig
    Fin Para
  Fin Si
Fin
```

4. Prueba de escritorio

Caso	n	Validación	Salida
Normal	7	válido	0 1 1 2 3 5 8
Normal	10	válido	0 1 1 2 3 5 8 13 21 34
Límite (mínimo)	1	válido	0
Límite	2	válido	0 1
Error	0	n < 1	Ingresa un valor entre 1 y 90
Error	-4	n < 1	Ingresa un valor entre 1 y 90
Error	91	n > 90	Ingresa un valor entre 1 y 90

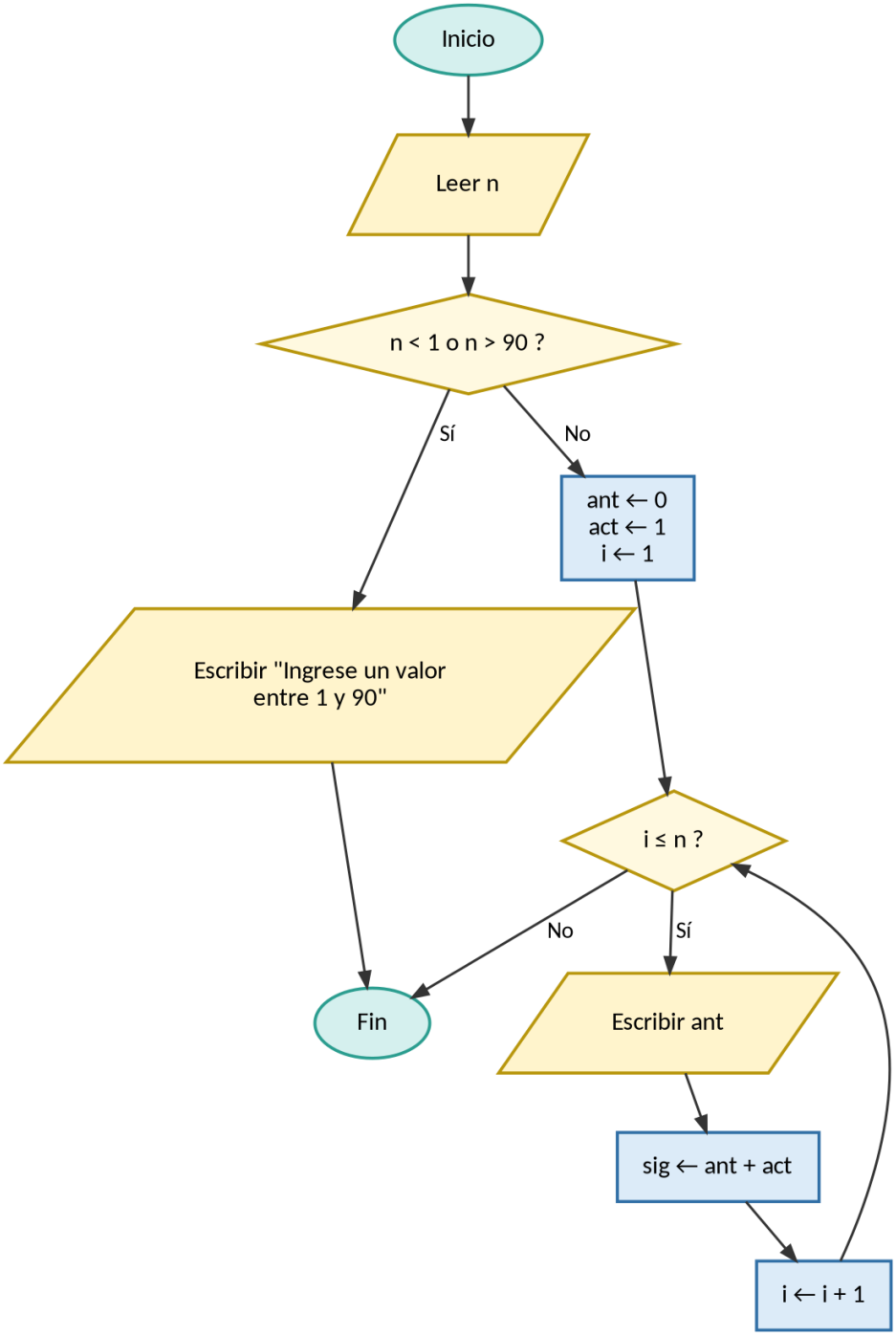
~~**Para pensar:** ¿Cuántas veces se repite tu ciclo para un número de k dígitos? Justifica tu respuesta.~~

Para pensar: ¿Cómo cambia tu algoritmo si solo debes mostrar el término número n y no toda la serie?

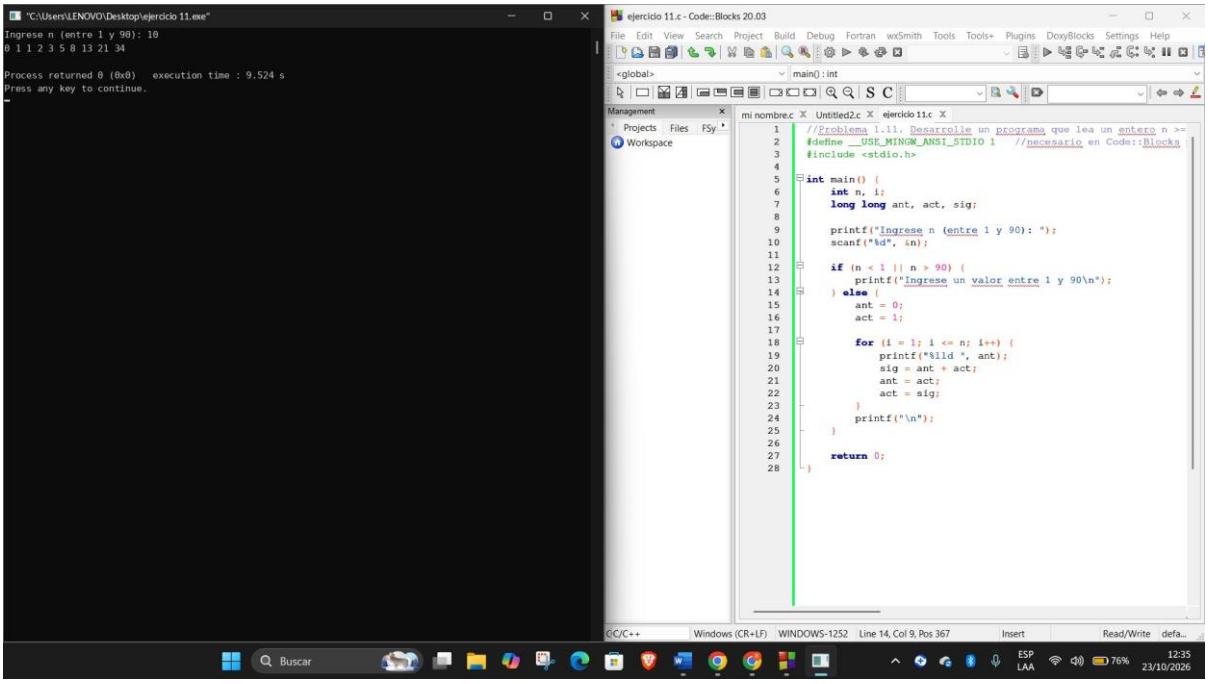
Se saca el Escribir fuera del ciclo y el ciclo se repite n-1 veces, porque el término 1 (el 0) ya está en ant al inicio. Al terminar, ant guardó el término número n y se escribe una sola vez. Las tres variables siguen siendo necesarias y no hace falta guardar la serie completa.

Ejemplo: n = 7 → 6 vueltas → ant = 8

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.12. Suma de dígitos

Desarrolla un programa que lea un entero positivo y calcule la suma de sus dígitos

1. Análisis del problema

Idea general: El último dígito de un número es su resto al dividirlo entre 10, y al hacer la división entera entre 10 ese dígito desaparece. Repitiendo esto se recorren todos los dígitos de derecha a izquierda y se van sumando.

Entradas: Un entero positivo n

Proceso: suma ← 0 y copia ← n. Mientras copia > 0: suma ← suma + (copia mod 10) y copia ← copia div 10. Se usa una copia para no perder el valor original de n.

Salida: Un mensaje con la suma de los dígitos de n

Restricción: n > 0

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Copia de n que se va reduciendo	copia	var	entero
Acumulador de la suma	suma	var	entero
Base decimal	10	cte	entero

Objeto	Nombre	Valor	Tipo
Cero (para validar)	0	cte	entero

3. Seudocódigo

```
Inicio
  Leer n
  Si n <= 0 entonces
    Escribir "Ingrese un entero positivo
  Si no
    suma ← 0
    copia ← n
    Mientras copia > 0 hacer
      suma ← suma + copia mod 10
      copia ← copia div 10
    Fin Mientras
    Escribir "La suma de los dígitos de ", n, " es ", suma
  Fin Si
Fin
```

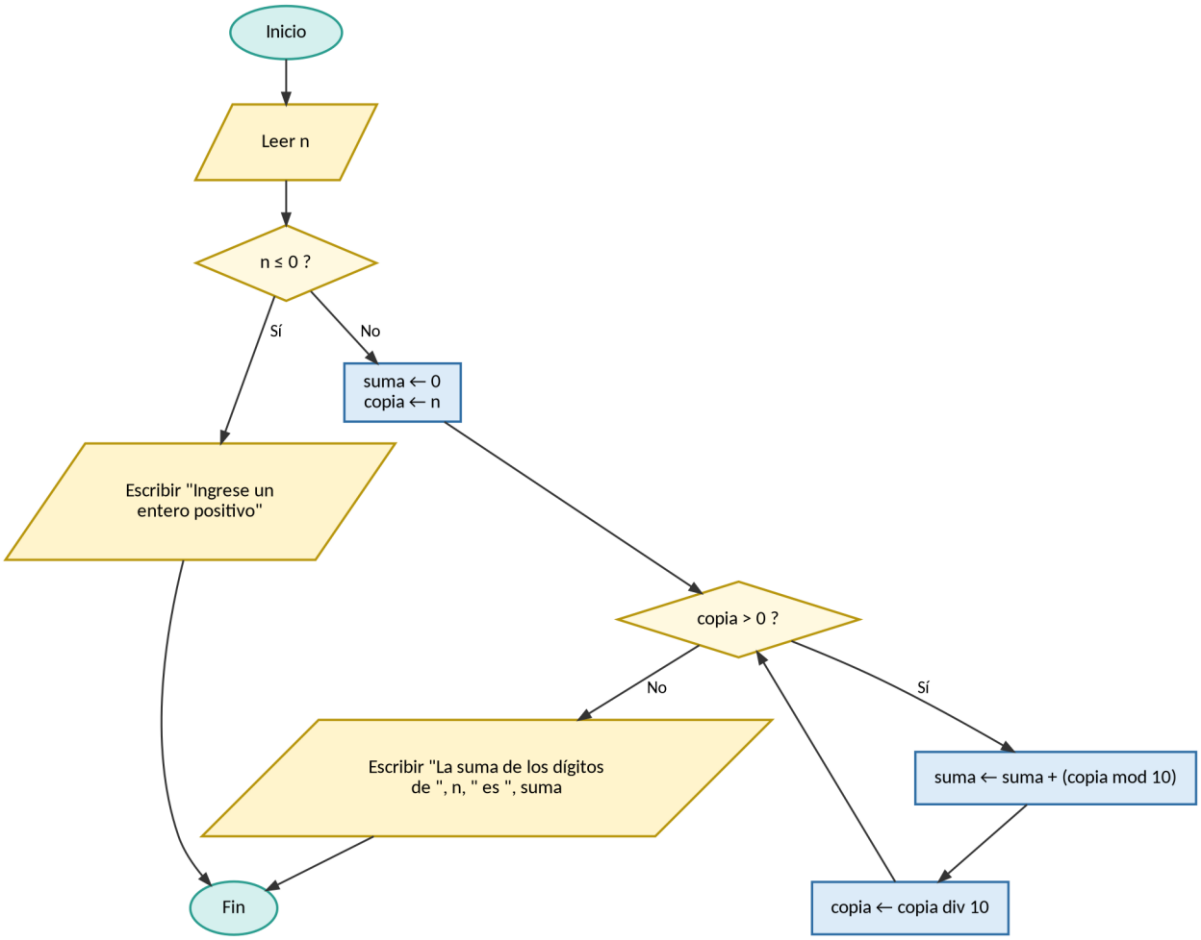
4. Prueba de escritorio

Caso	n	n ≤ 0	vueltas	suma	Salida
Normal	4825	No	4	19	La suma de los dígitos de 4825 es 19
Un solo dígito	7	No	1	7	La suma de los dígitos de 7 es 7
Con cero	100	No	3	1	La suma de los dígitos de 100 es 1
Todos nueves	99999	No	5	45	La suma de los dígitos de 99999 es 45
Error	0	Sí	-	-	Ingrese un entero positivo
Error	-15	Sí	-	-	Ingrese un entero positivo

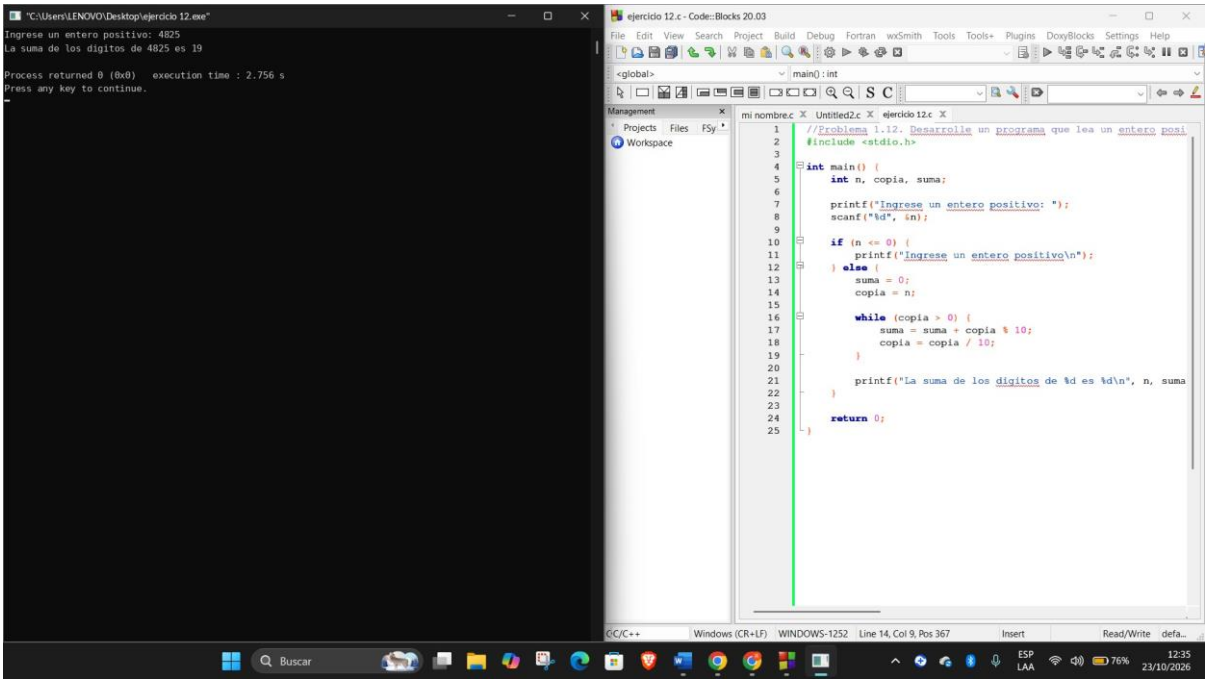
Para pensar: ¿Cuántas veces se repite tu ciclo para un número de k dígitos? Justifica tu respuesta.
Se repite exactamente k veces. En cada vuelta, copia div 10 elimina un solo dígito (el último). Un número de k dígitos cumple 10^(k-1) ≤ n < 10^k, así que después de k divisiones entre 10 el valor llega a 0 y la condición copia > 0 deja de cumplirse.

Ejemplo: 4825 (k = 4) → 4 vueltas; 7 (k = 1) → 1 vuelta; 100 (k = 3) → 3 vueltas, igual que se ve en la prueba de escritorio.

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.13. Número capicúa

Desarrolla un programa que lea un entero positivo y determine si es capicúa, es decir, si se lee igual de izquierda a derecha que de derecha a izquierda.

1. Análisis del problema

Idea general: Un número es capicúa si al invertir sus dígitos se obtiene el mismo número; los dígitos se extraen de derecha a izquierda y se van colocando en un número nuevo, de modo que este queda invertido

Entradas: Un entero positivo n

Proceso: copia ← n e invertido ← 0. Mientras copia > 0: invertido ← invertido * 10 + (copia mod 10) y copia ← copia div 10. Al final se compara invertido con n

Salida: Un mensaje que indica si n es capicúa o no

Restricción: n > 0 y a lo sumo 18 dígitos, para que el número invertido quepa en un long long

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero largo
Copia de n que se va reduciendo	copia	var	entero largo
Número invertido	invertido	var	entero largo
Base decimal	10	cte	entero
Cero (para validar)	0	cte	entero

3. Seudocódigo

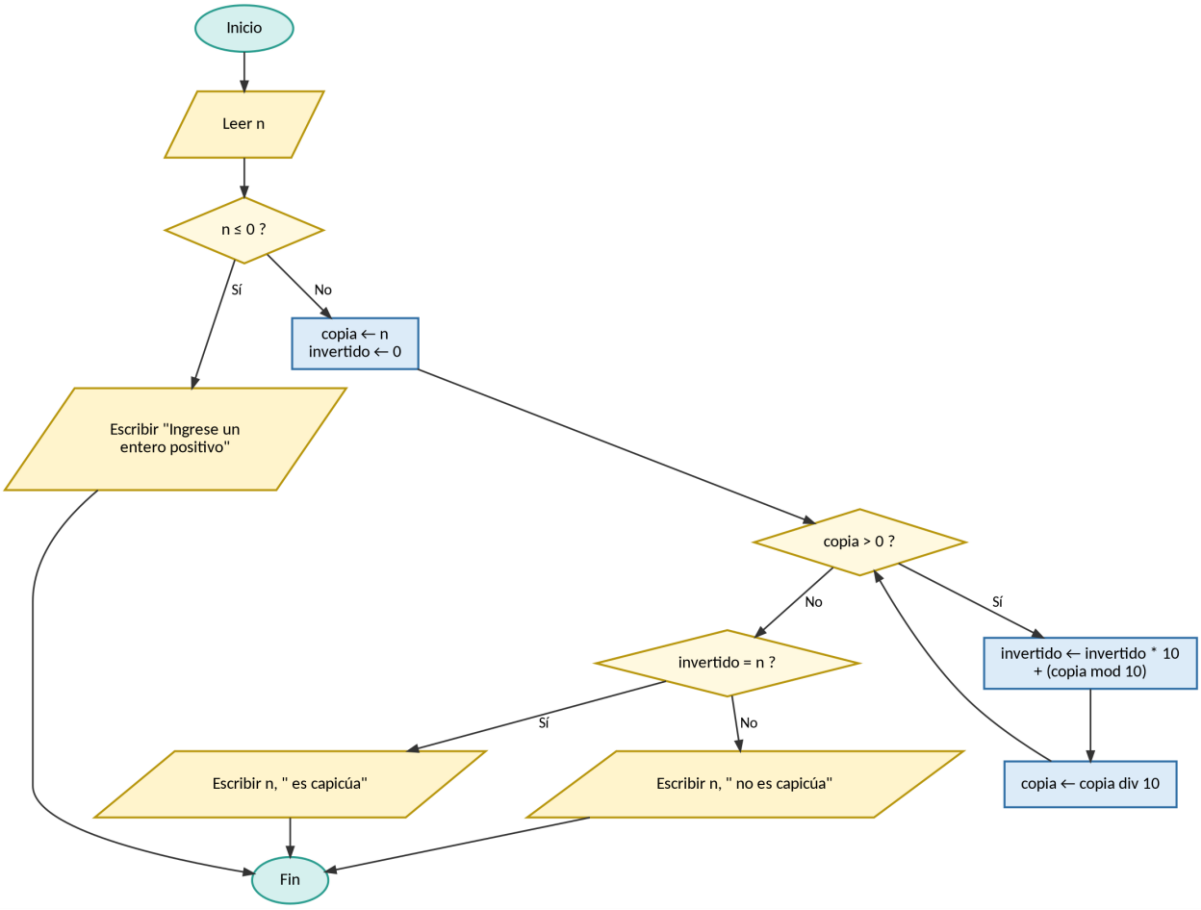
```
Inicio
  Leer n
  Si n <= 0 entonces
    Escribir "Ingrese un entero positivo
  Sino
    copia ← n
    invertido ← 0
    Mientras copia > 0 hacer
      invertido ← invertido * 10 + copia mod 10
      copia ← copia div 10
    Fin mientras
    Si invertido = n entonces
      Escribir n, " es capicúa"
    Sino
      Escribir n, " no es capicúa"
    Fin Si
```

```
  Fin Si
Fin
```

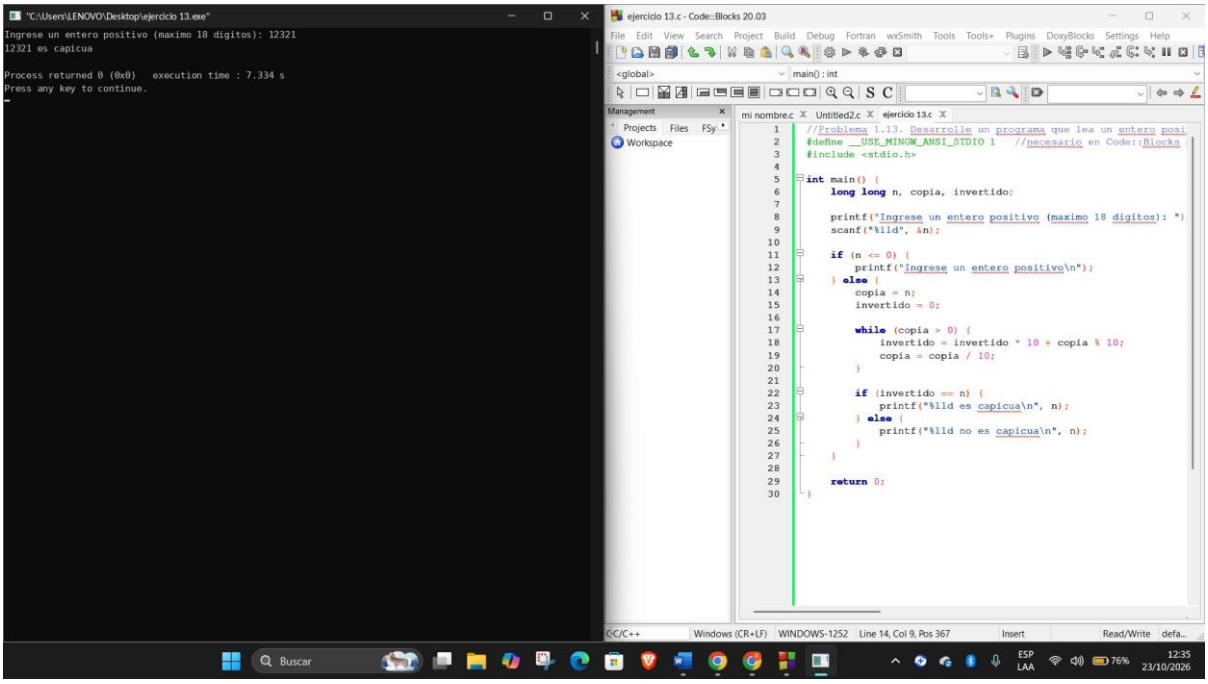
4. Prueba de escritorio

Caso	n	n ≤ 0	invertido	invertido = n	Salida
Normal (impar de dígitos)	12321	No	12321	Sí	12321 es capicúa
Normal (no capicúa)	1231	No	1321	No	1231 no es capicúa
Par de dígitos	1221	No	1221	Sí	1221 es capicúa
Límite (1 dígito)	7	No	7	Sí	7 es capicúa
Termina en 0	1210	No	121	No	1210 no es capicúa
Termina en 0	100	No	1	No	100 no es capicúa
Error	0	Sí	-	-	Ingrese un entero positivo
Error	-5	Sí	-	-	Ingrese un entero positivo

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.14. Máximo común divisor (algoritmo de Euclides)

Desarrolla un programa que lea dos enteros positivos a y b y calcule su máximo común divisor con el algoritmo de Euclides: mientras b ≠ 0, reemplaza (a, b) por (b, a % b). Al terminar, a es el MCD

1. Análisis del problema

Idea general: El MCD es el mayor número que divide exactamente a los dos. Euclides lo encuentra sin probar divisores: reemplaza el par (a, b) por (b, resto de a entre b) hasta que el segundo número sea 0.

Entradas: Dos enteros positivos a y b

Proceso: mientras b ≠ 0: r ← a mod b; a ← b; b ← r. Cuando b = 0, el valor de a es el MCD.

Salida: Un mensaje con el MCD

Restricción: a > 0 y b > 0. No importa si a < b: en la primera vuelta el par se invierte solo (a mod b = a).

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (primer número)	a	var	entero
Dato de entrada (segundo número)	b	var	entero
Resto de la división	r	var	entero

Objeto	Nombre	Valor	Tipo
Cero (condición de parada y validación)	0	cte	entero

3. Seudocódigo

```
Inicio
  Leer a, b
  Si a <= 0 o b <= 0 entonces
    Escribir "Ingrese dos enteros positivos"
  Sino
    Mientras b <> 0 hacer
      r ← a mod b
      a ← b
      b ← r
    Fin mientras
    Escribir "El MCD es ", a
  Fin Si
Fin
```

4. Prueba de escritorio

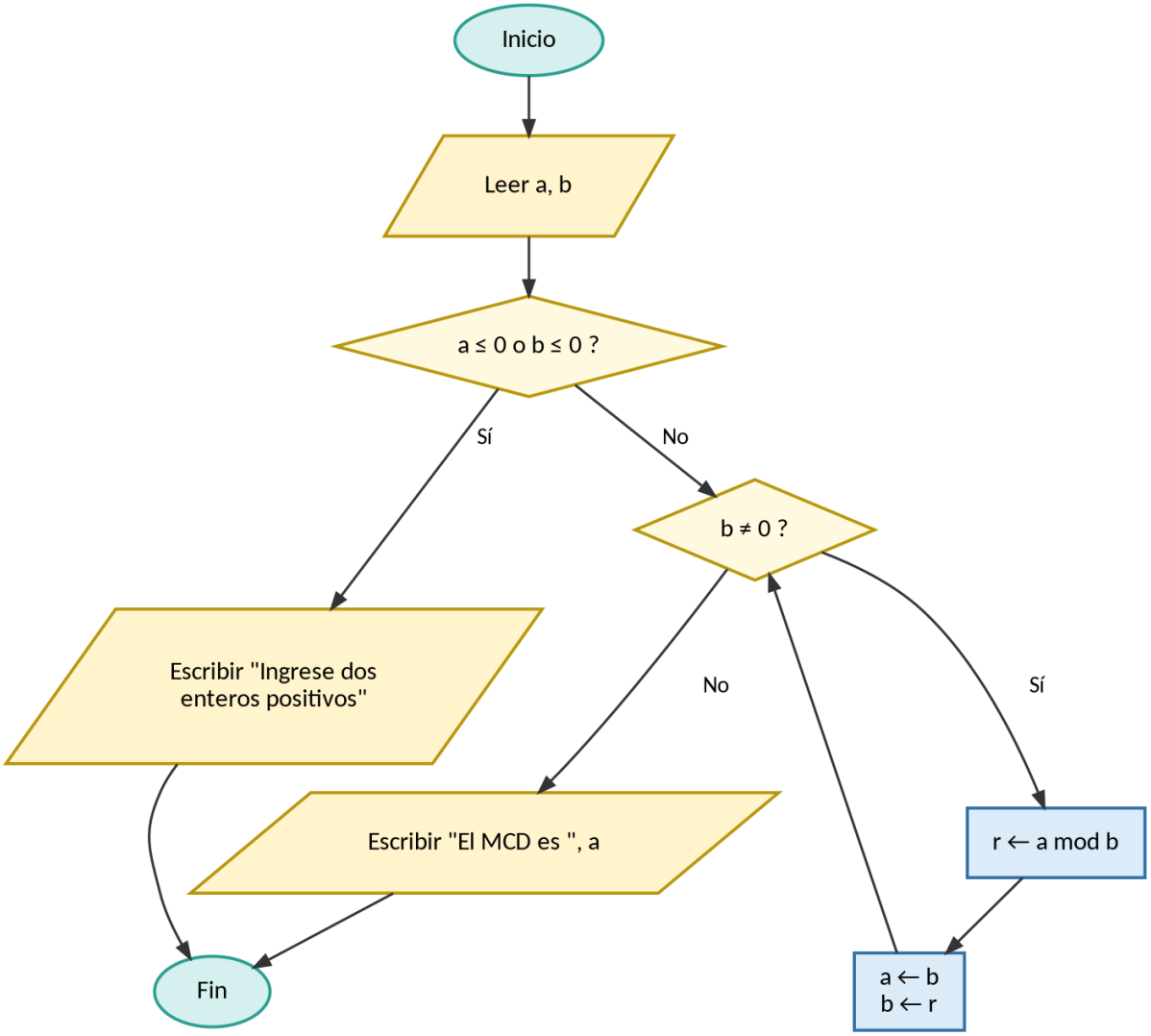
Caso	a	b	vueltas	Salida
Normal	48	18	3	El MCD es 6
Normal	100	75	2	El MCD es 25
Primos entre sí	17	5	3	El MCD es 1
Iguales	10	10	1	El MCD es 10
a menor que b	18	48	4	El MCD es 6
Límite (b = 1)	7	1	1	El MCD es 1
Error	0	5	-	Ingrese dos enteros positivos
Error	5	-2	-	Ingrese dos enteros positivos

Para pensar: Compara este método con probar todos los divisores desde 1 hasta el menor de los dos números. ¿Cuál hace menos operaciones y por qué?

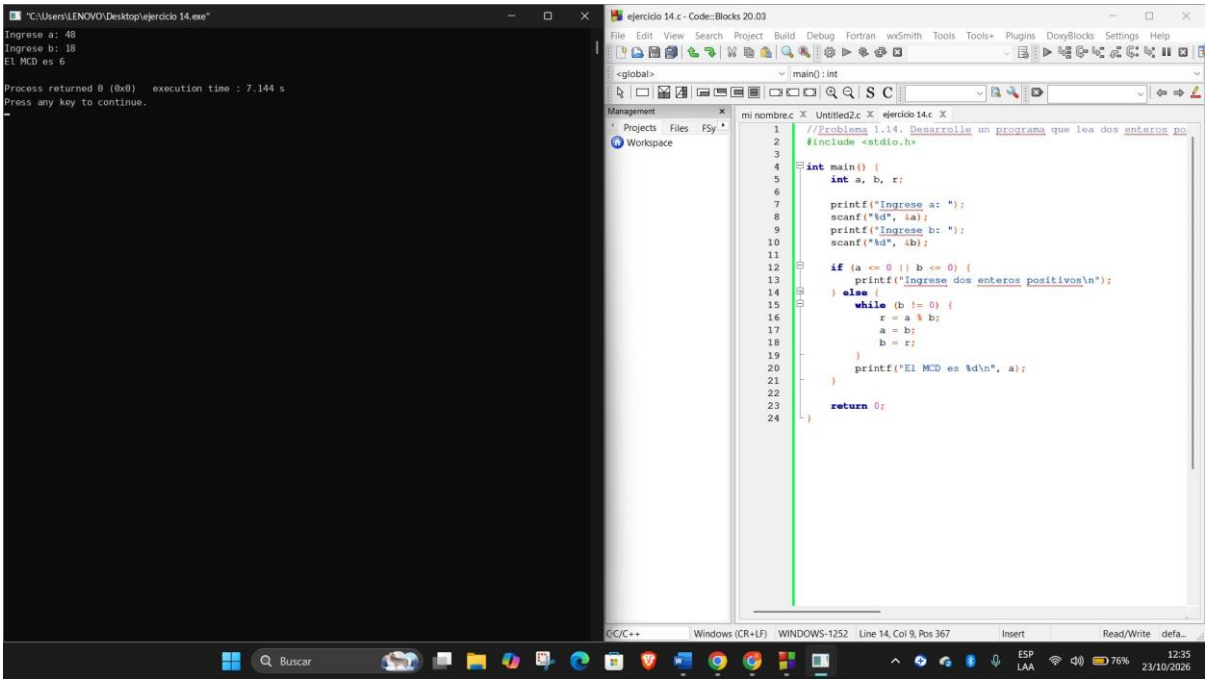
Euclides hace menos operaciones. Para (48, 18), probar todos los divisores desde 1 hasta 18 hace 18 vueltas (cada una con 2 restos), mientras que Euclides hace solo 3 vueltas (48 mod 18 = 12, 18 mod 12 = 6, 12 mod 6 = 0). La diferencia se vuelve enorme con números grandes: para (1 000 000, 999 999), el método de los divisores hace 999 999 vueltas y Euclides solo 2 (1 000 000 mod 999 999 = 1 y 999 999 mod 1 = 0).

La razón es que en cada paso Euclides reemplaza los números por otros mucho más pequeños.

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.15. Año bisiesto

Desarrolla un programa que lea un año y determine si es bisiesto. Un año es bisiesto si es divisible entre 4, excepto los años divisibles entre 100, que solo lo son si también son divisibles entre 400.

1. Análisis del problema

Idea general: Se aplica la regla del calendario: un año es bisiesto si es múltiplo de 4, pero los múltiplos de 100 solo son bisiestos cuando además son múltiplos de 400

Entradas: Un entero, el año

Proceso: El año es bisiesto si (año mod 4 = 0 Y año mod 100 ≠ 0) O (año mod 400 = 0). Primera parte: múltiplo de 4 que no es múltiplo de 100. Segunda parte: la excepción de los múltiplos de 400.

Salida: Un mensaje que indica si el año es bisiesto o no.

Restricción: año > 0

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (año)	anio	var	Entero
Divisor para el ciclo normal	4	cte	Entero
Divisor para los siglos	100	cte	Entero
Divisor para la excepción	400	cte	Entero
Cero (para comparar el resto y validar)	0	cte	Entero

3. Seudocódigo

```
Inicio
  Leer anio
  Si anio <= 0 entonces
    Escribir "Año no válido"
  Sino
    Si (anio mod 4 = 0 Y anio mod 100 <> 0) O anio mod 400 = 0 entonces
      Escribir anio, " es bisiesto"
    Sino
      Escribir anio, " no es bisiesto"
    Fin Si
  Fin Si
Fin
```

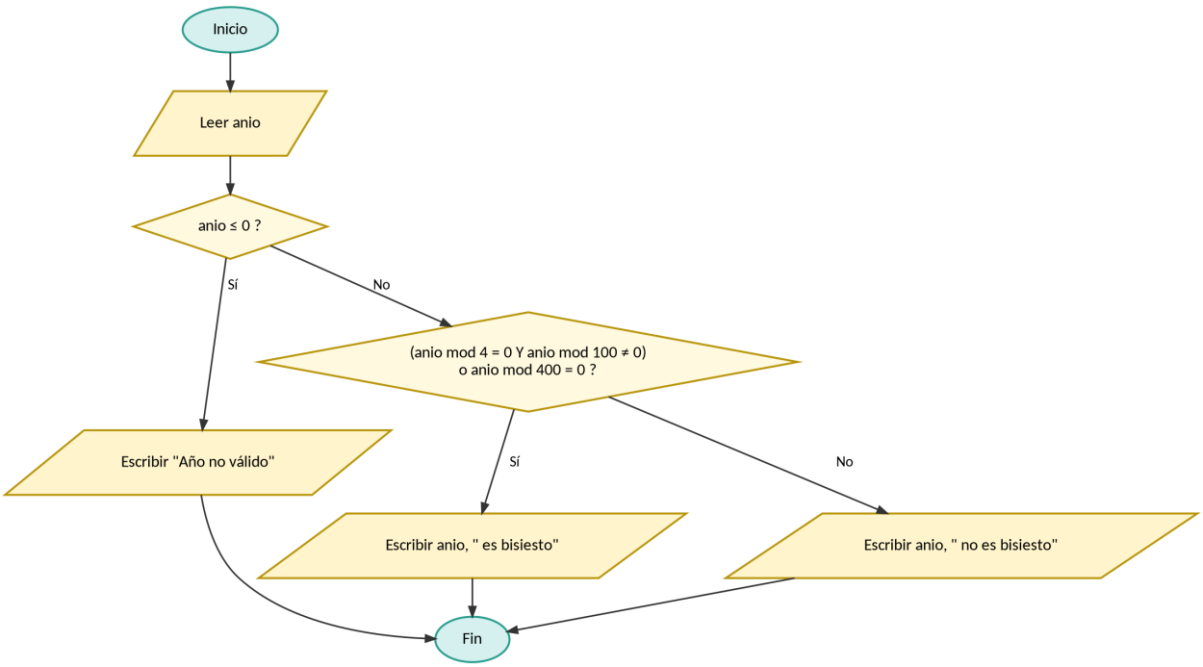
4. Prueba de escritorio

Caso	Año	mod 4	mod 100	mod 400	Bisiesto	Salida
Normal (Sí)	2024	0	24	24	Sí	2024 es bisiesto
Normal (No)	2023	3	23	23	No	2023 no es bisiesto
Siglo (no)	1900	0	0	300	No	1900 no es bisiesto
Siglo (sí)	2000	0	0	0	Sí	2000 es bisiesto
Siglo (no)	2100	0	0	100	No	2100 no es bisiesto
Siglo (sí)	2400	0	0	0	Sí	2400 es bisiesto
Error	0	-	-	-	-	Año no válido
Error	-4	-	-	-	-	Año no válido

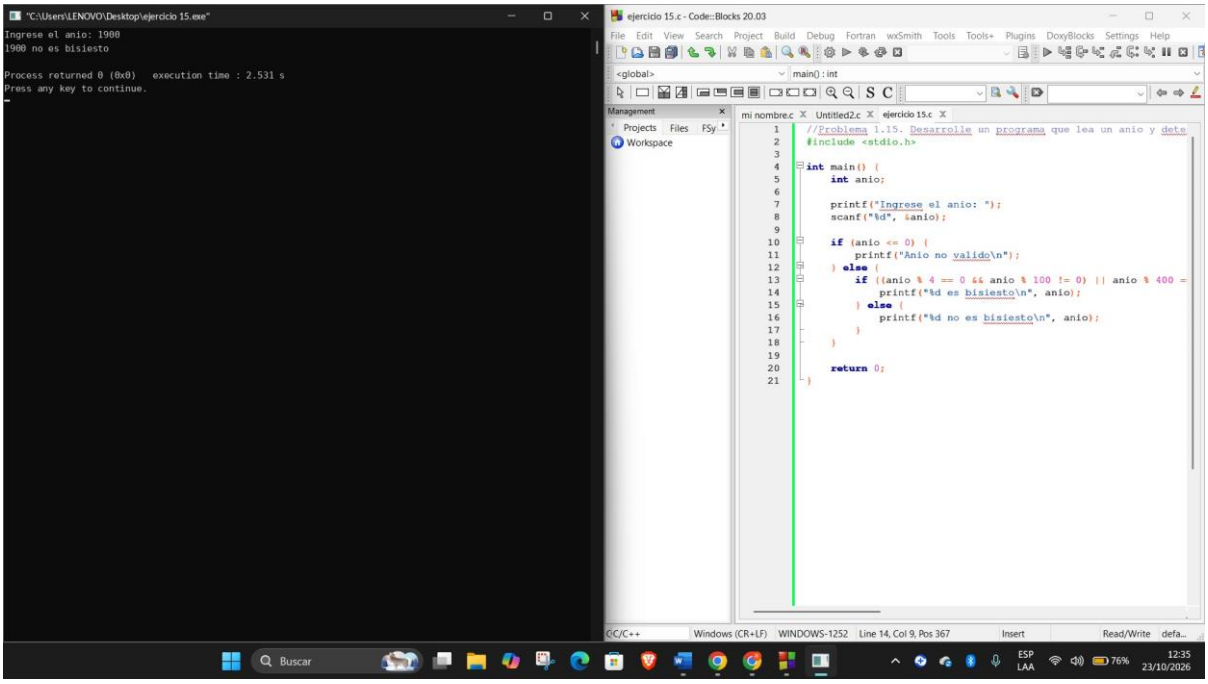
Para pensar: ¿Es equivalente escribir las condiciones en otro orden? Prueba con 1900 y 2000 para comprobarlo.

En una sola condición con paréntesis, el orden de los operandos no cambia el resultado: con 1900 da falso y con 2000 verdadero. Con condiciones anidadas sí importa: si solo se pregunta mod 4 = 0 y se responde «bisiesto», 1900 saldría mal. Hay que preguntar primero por las excepciones (mod 400, luego mod 100) y al final por mod 4.

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.16. Número perfecto

Desarrolla un programa que lea un entero positivo y determine si es perfecto, es decir, si es igual a la suma de sus divisores propios (los menores que él).

1. Análisis del problema

Idea general: Un número es perfecto si es igual a la suma de los divisores que son menores que él. Se revisan todos los números de 1 a n − 1 y se suman los que dividen exactamente a n.

Entradas: Un entero positivo n

Proceso: suma ← 0. Para d = 1, 2, ..., n − 1: si n mod d = 0, suma ← suma + d. Se compara con n

Salida: Un mensaje que indica si n es perfecto o no

Restricción: n > 0. Para n = 1 no hay divisores propios (suma = 0 ≠ 1), así que no es perfecto

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Posible divisor	d	var	entero
Suma de divisores propios	suma	var	entero
Cero (para validar)	0	cte	entero

3. Seudocódigo

```
Inicio
  Leer n
  Si n <= 0 entonces
    Escribir "Ingrese un entero positivo"
  Si no
    suma ← 0
    Para d ← 1 hasta n - 1 hacer
      Si n mod d = 0 entonces
        suma ← suma + d
      Fin Si
    Fin Para
    Si suma = n entonces
      Escribir n, " es perfecto"
    Sino
      Escribir n, " no es perfecto"
    Fin Si
  Fin Si
Fin
```

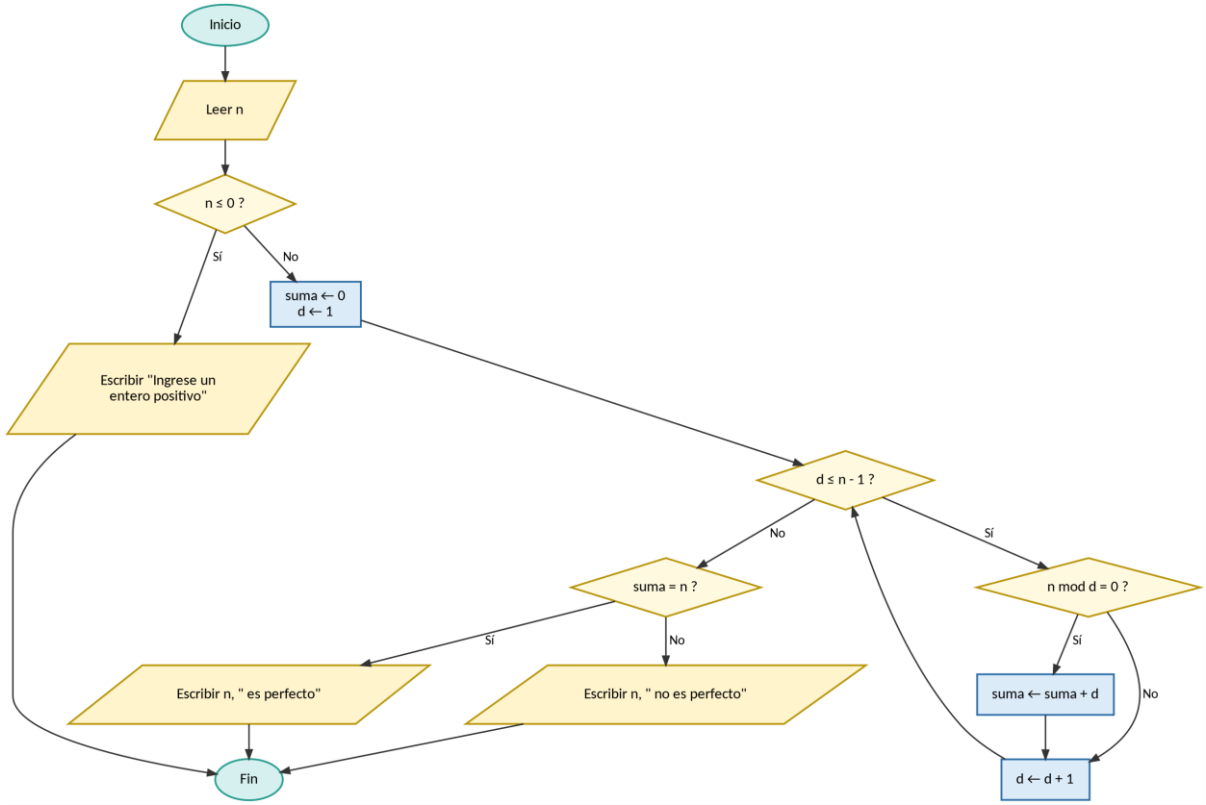
4. Prueba de escritorio

Caso	n	Divisores propios	suma	suma = n?	Salida
Normal (perfecto)	28	1, 2, 4, 7, 14	28	sí	28 es perfecto
Normal (perfecto)	6	1, 2, 3	6	sí	6 es perfecto
Normal (no perfecto)	12	1, 2, 3, 4, 6	16	no	12 no es perfecto
Normal (no perfecto)	10	1, 2, 5	8	no	10 no es perfecto
Grande	496	1, 2, 4, 8, 16, 31, 62, ...	496	sí	496 es perfecto
Límite	1	ninguno	0	no	1 no es perfecto
Error	0	-	-	-	Ingrese un entero +
Error	-6	-	-	-	Ingrese un entero +

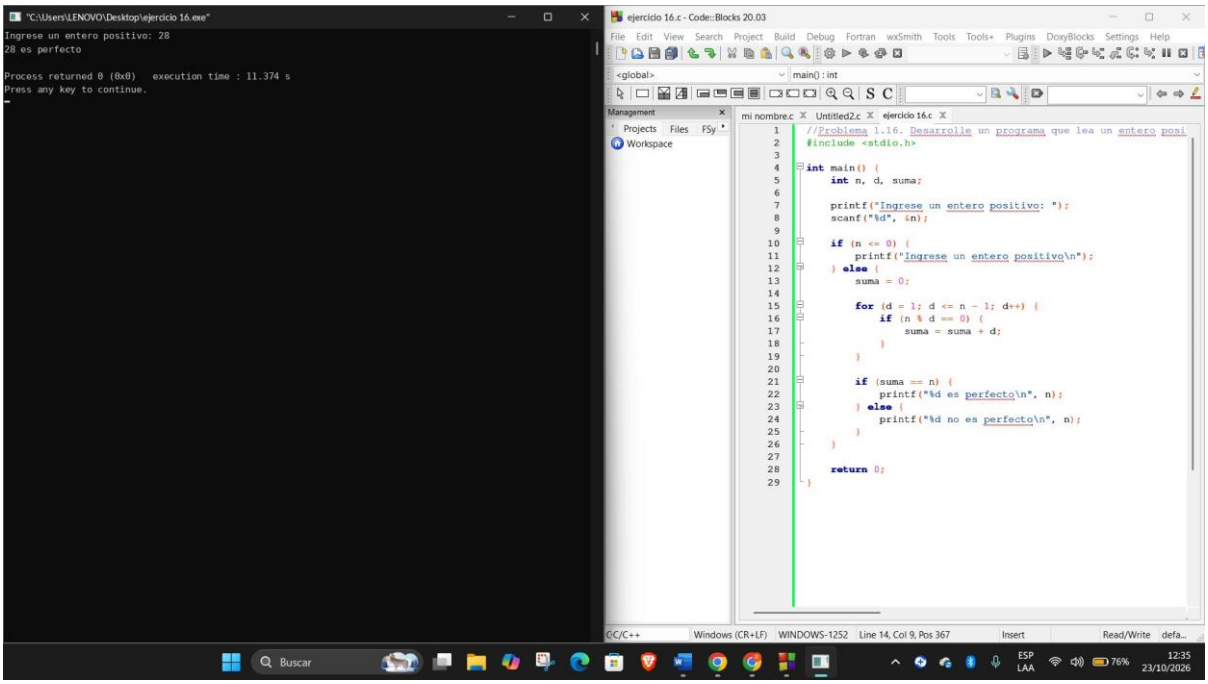
Para pensar: ¿Hasta qué valor es realmente necesario recorrer los divisores? Explica cómo reducirías el número de vueltas.

Basta hasta n/2, porque ningún divisor propio es mayor: para 28, de 27 a 14 vueltas. Mejor aún, hasta √n sumando las parejas (d, n/d) sin contar n: para 28, d = 1..5 → 1 + 2 + 14 + 4 + 7 = 28; para 496, de 495 a 22 vueltas.

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.17. Conversión de decimal a binario

Desarrolla un programa que lea un entero positivo en base 10 y muestre su representación en binario.

1. Análisis del problema

Idea general: Para pasar a binario se divide entre 2 una y otra vez; los restos (0 o 1) son los dígitos binarios, pero salen de derecha a izquierda. Para no necesitar arreglos, cada resto se coloca en su posición dentro de un número entero usando pesos de 1, 10, 100, ..., y así el resultado queda en el orden correcto.

Entradas: Un entero positivo n en base 10

Proceso: copia ← n, binario ← 0, peso ← 1. Mientras copia > 0: resto ← copia mod 2; binario ← binario + resto * peso; peso ← peso * 10; copia ← copia div 2.

Salida: Un mensaje con n y su representación en binario

Restricción: 1 ≤ n ≤ 262 143 (el resultado tiene a lo sumo 18 cifras, que caben en un long).

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Copia de n que se va dividiendo	copia	var	entero largo
Resto de dividir entre 2	resto	var	entero
Número binario construido	binario	var	entero largo
Posición (peso)	peso	var	entero largo
Máximo n permitido	262143	cte	entero

3. Seudocódigo

```
Inicio
  Leer n
  Si n < 1 o n > 262143 entonces
    Escribir "Ingrese un entero entre 1 y 262143"
  Sino
    copia ← n
    binario ← 0
    peso ← 1
    Mientras copia > 0 hacer
      resto ← copia mod 2
      binario ← binario + resto * peso
      peso ← peso * 10
      copia ← copia div 2
    Fin Mientras
    Escribir n, " es binario en ", binario
  Fin Si
Fin
```

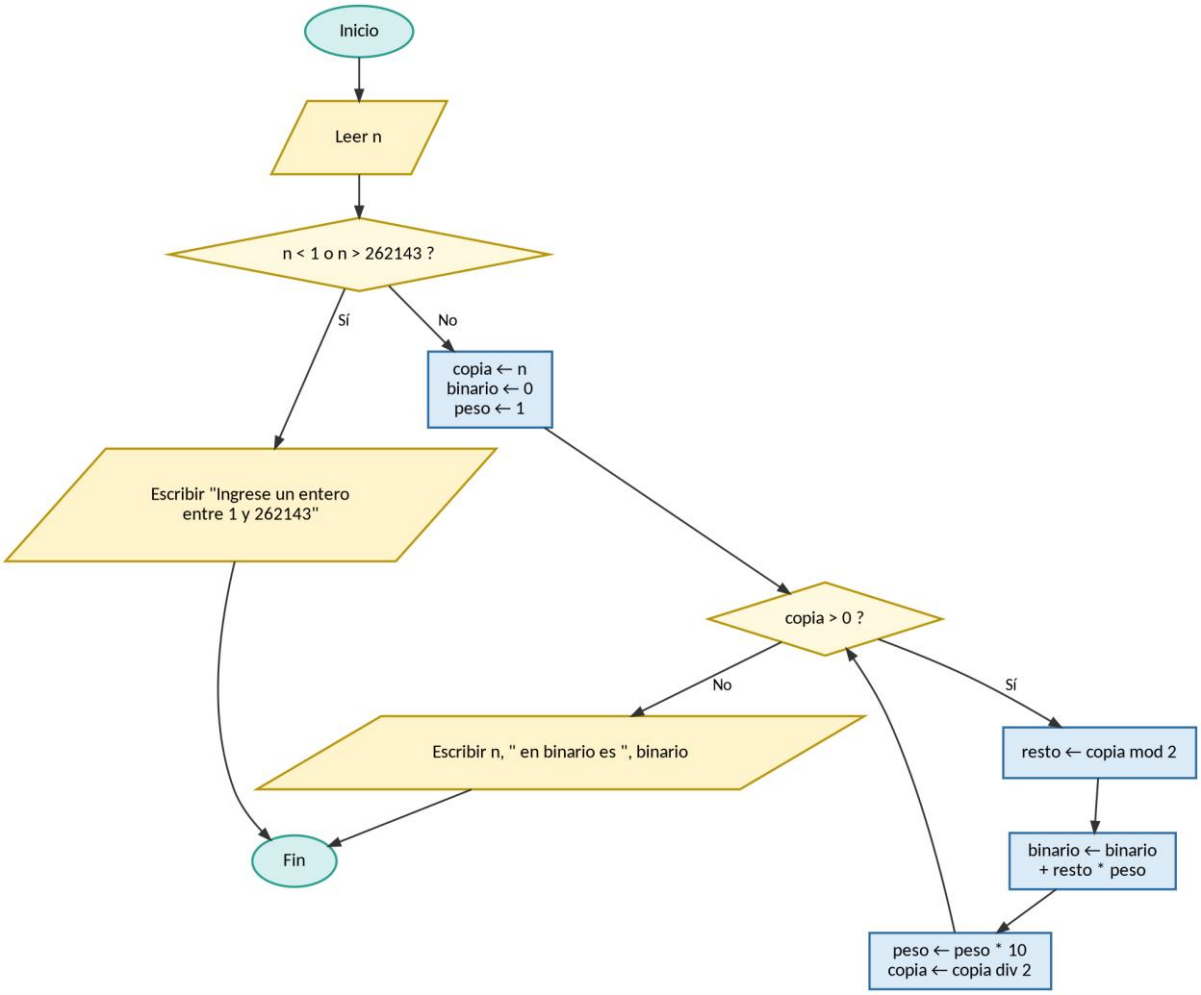
4. Prueba de escritorio

Caso	n	Restos obtenidos	Salida
Normal	13	1, 0, 1, 1	13 en binario es 1101
Normal	255	1, 1, 1, 1, 1, 1, 1, 1	255 en binario es 11111111
Potencia de 2	8	0, 0, 0, 1	8 en binario es 1000
Límite (mínimo)	1	1	1 en binario es 1
Límite	2	0, 1	2 en binario es 10
Límite (máximo)	262143	18 unos	262143 en binario es 111111111111111111
Error	0	-	Ingrese un entero entre 1 y 262143
Error	300000	-	Ingrese un entero entre 1 y 262143

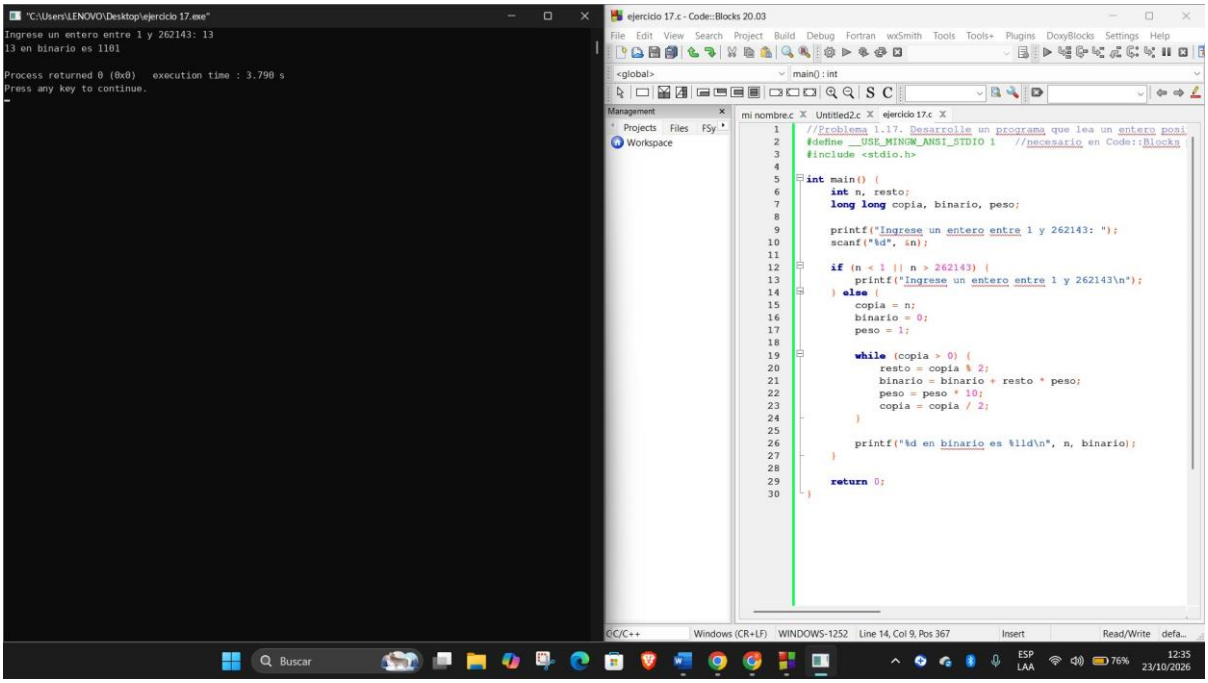
Para pensar: Los restos se obtienen en orden inverso al que se leen. ¿Cómo resolverías ese detalle sin usar arreglos?

Acumular los restos de un número con peso ×10 en cada vuelta (13 → restos 1, 0, 1, 1 → 1101); queda en orden correcto, con límite de 18 cifras. O empezar por la mayor potencia de 2 ≤ n, p, y escribir 1 si n ≥ p (restando p) o 0 si no, con p ← p div 2; para 13: 1, 1, 0, 1.

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.18. Triángulo de asteriscos

Desarrolla un programa que lea un entero n ≥ 1 y dibuje por pantalla un triángulo rectángulo de n filas, donde la fila i tiene i asteriscos.

1. Análisis del problema

Idea general: La fila i debe tener i asteriscos. Se usan dos ciclos: el de afuera recorre las filas y el de adentro escribe los asteriscos de cada fila; al terminar cada fila se baja de línea.

Entradas: Un entero n, la cantidad de filas.

Proceso: para i = 1, ..., n: para j = 1, ..., i se escribe un asterisco; al terminar el ciclo interno se escribe un salto de línea.

Salida: El triángulo de n filas hecho con asteriscos

Restricción: n ≥ 1

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (número de filas)	n	var	entero
Contador de filas	i	var	entero
Contador de asteriscos de la fila	j	var	entero
Carácter que se dibuja	*	cte	carácter
Uno (límite de validación)	1	cte	entero

3. Seudocódigo

```
Inicio
  Leer n
  Si n < 1 entonces
    Escribir "Ingrese un entero mayor o igual que 1"
  Si no
    Para i ← 1 hasta n hacer
      Para j ← 1 hasta i hacer
        Escribir "*"
      Fin Para
      Escribir salto de línea
    Fin Para
  Fin Si
Fin
```

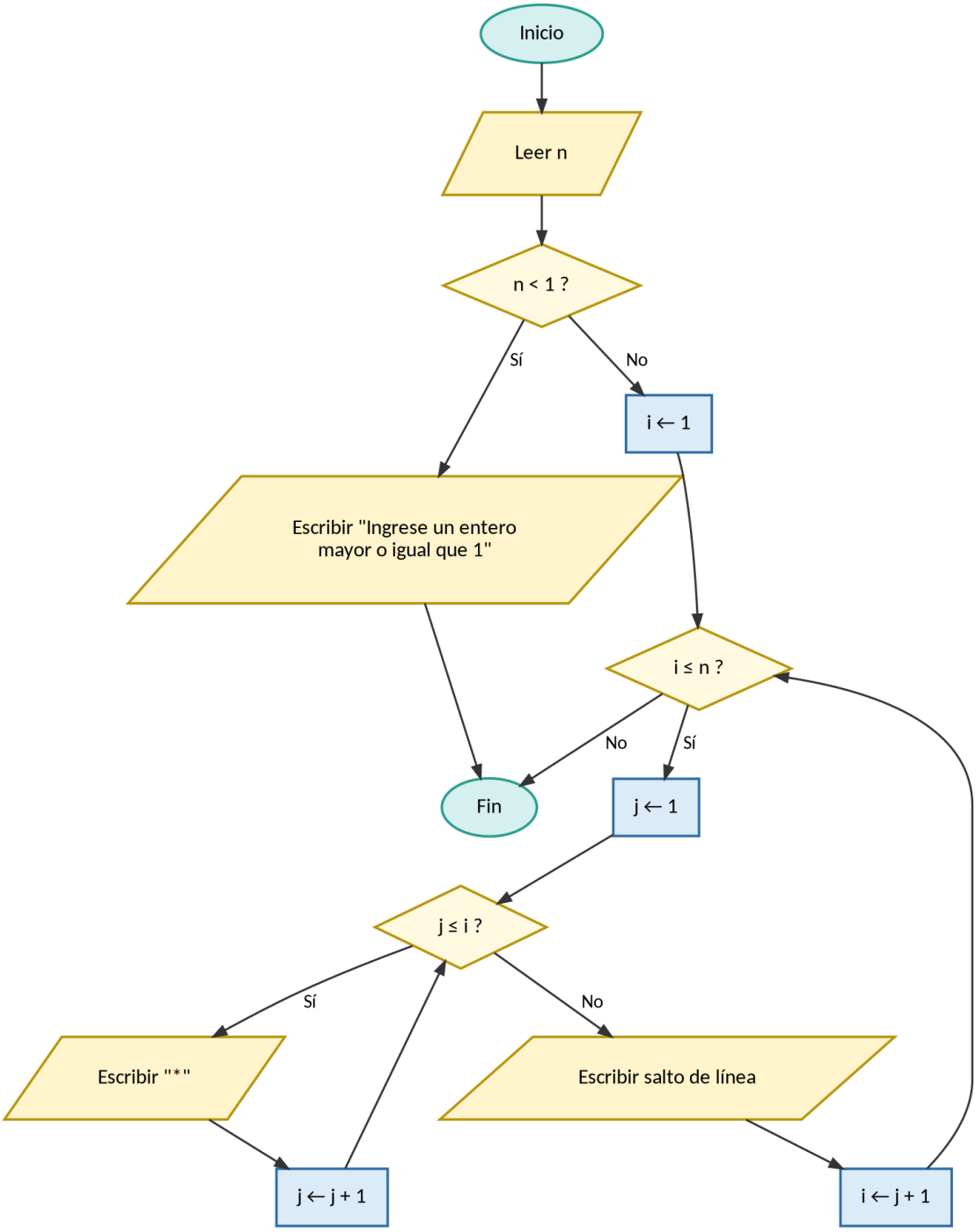
4. Prueba de escritorio

Caso	n	Salida
Normal	4	* ** *** ****
Normal	3	* ** ***
Límite (mínimo)	1	*
Error	0	Ingrese un entero mayor o igual que 1
Error	-2	Ingrese un entero mayor o igual que 1

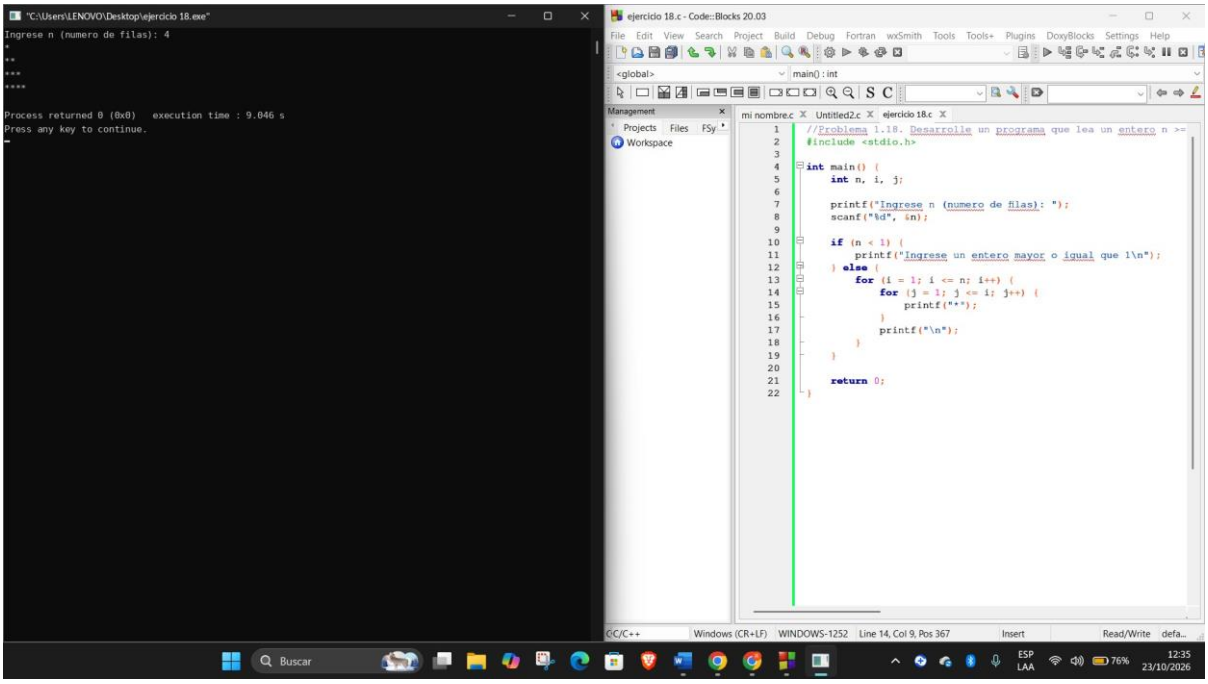
Para pensar: Si ahora quisieras un triángulo centrado (una pirámide), ¿qué parte de tu algoritmo cambiarías y qué parte se mantendría?

Se mantienen el ciclo externo y el salto de línea. Cambia el ciclo interno: antes se escriben n − i espacios y los asteriscos pasan de i a 2i − 1 (n = 4: la fila 1 tiene 3 espacios y 1 asterisco; la fila 4, 0 espacios y 7 asteriscos).

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)



Problema 1.19. Segundo mayor de una secuencia

Desarrolla un programa que lea números enteros positivos hasta que se introduzca un 0, muestre el mayor y el segundo mayor de los valores ingresados. Si hay menos de dos valores, debe informarlo.

1. Análisis del problema

Idea general: Se leen números hasta que llegue un 0 (centinela). Mientras se leen, solo se recuerdan dos valores: el mayor y el segundo mayor.

Entradas: Enteros positivos x, hasta que se ingrese 0

Proceso: por cada x > 0: se cuenta (cont ← cont + 1). Si x > mayor, el mayor anterior pasa a ser el segundo y x es el nuevo mayor. Si no, y x > segundo, entonces x es el nuevo segundo

Salida: El mayor y el segundo mayor, o un mensaje si se ingresaron menos de dos valores

Restricción: los valores deben ser positivos; el 0 termina la lectura y un negativo se ignora con un aviso

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	x	var	entero
Mayor valor leído	mayor	var	entero
Segundo mayor valor leído	segundo	var	entero

Objeto	Nombre	Valor	Tipo
Cantidad de valores válidos	cont	var	entero
Cero (centinela y valor inicial)	0	cte	entero

3. Seudocódigo

```
Inicio
  mayor ← 0
  segundo ← 0
  cont ← 0
  Leer x
  Mientras x <> 0 hacer
    Si x < 0 entonces
      Escribir "Valor negativo ignorado"
    Sino
      cont ← cont + 1
      Si x > mayor entonces
        segundo ← mayor
        mayor ← x
      Sino
        Si x > segundo entonces
          segundo ← x
        Fin Si
      Fin Si
    Fin Si
    Leer x
  Fin Mientras
  Si cont < 2 entonces
    Escribir "Se necesitan al menos dos valores"
  Sino
    Escribir "Mayor: ", mayor
    Escribir "Segundo mayor: ", segundo
  Fin Si
Fin
```

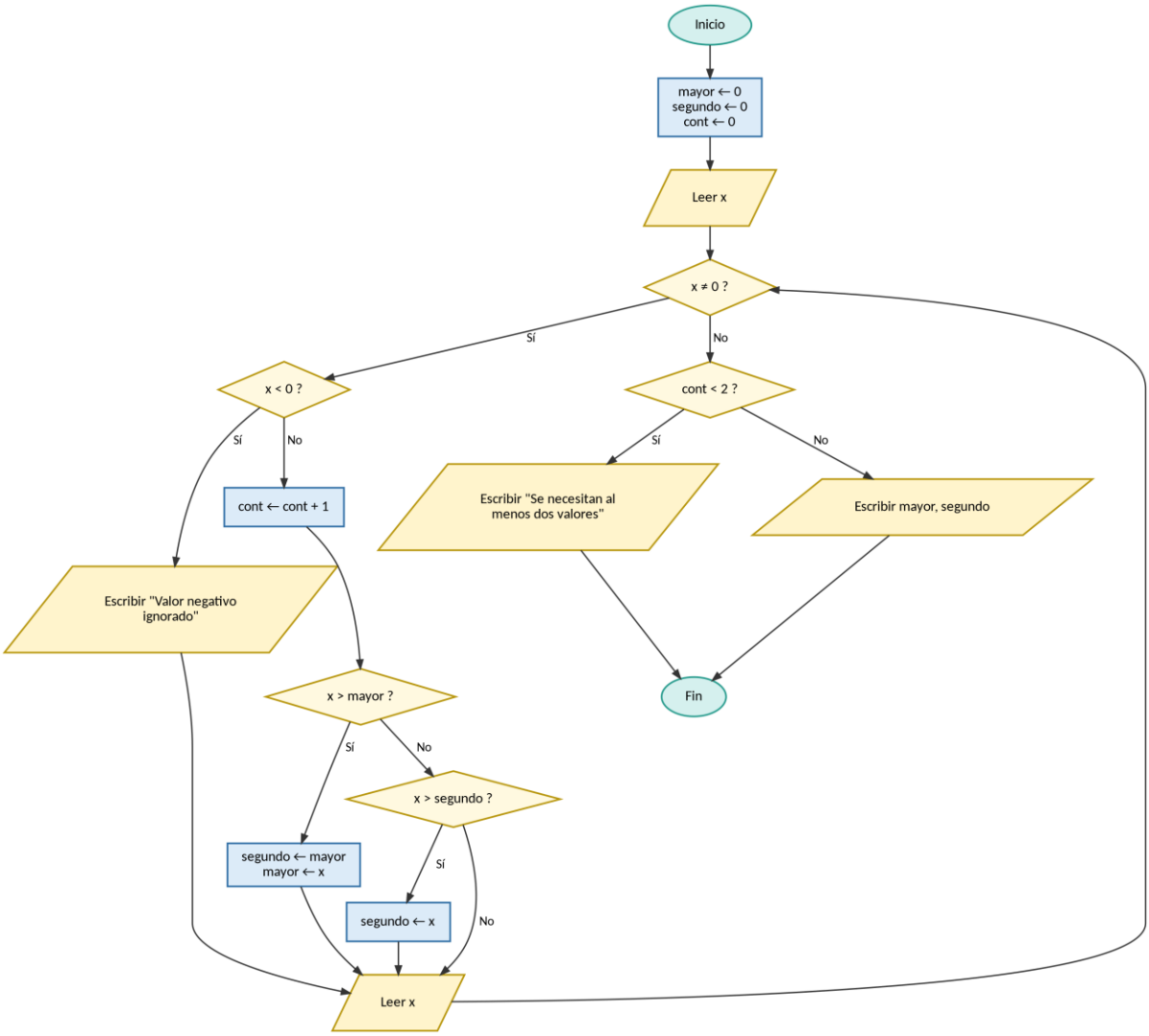
4. Prueba de escritorio

Caso	Datos ingresados	cont	Salida
Normal	7, 3, 9, 5, 0	4	Mayor: 9 / Segundo mayor: 7
Creciente	1, 2, 3, 0	3	Mayor: 3 / Segundo mayor: 2
Valor repetido	9, 9, 5, 0	3	Mayor: 9 / Segundo mayor: 9
Con negativo	-3, 6, 2, 0	2	Valor negativo ignorado / 6 / 2
Límite (un solo valor)	4, 0	1	Se necesitan al menos dos valores
Límite (sin valores)	0	0	Se necesitan al menos dos valores

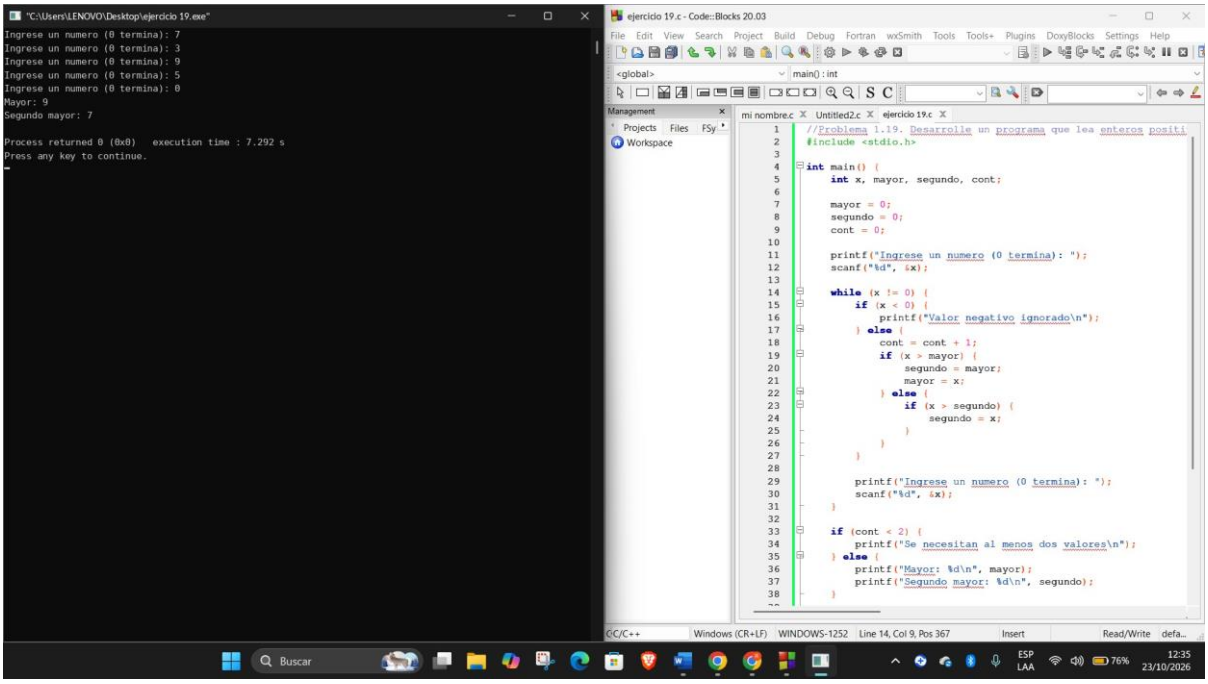
Para pensar: ¿Qué debería pasar si el valor mayor se repite (por ejemplo 9, 9, 5)? Decide, justifica y prueba tu decisión.

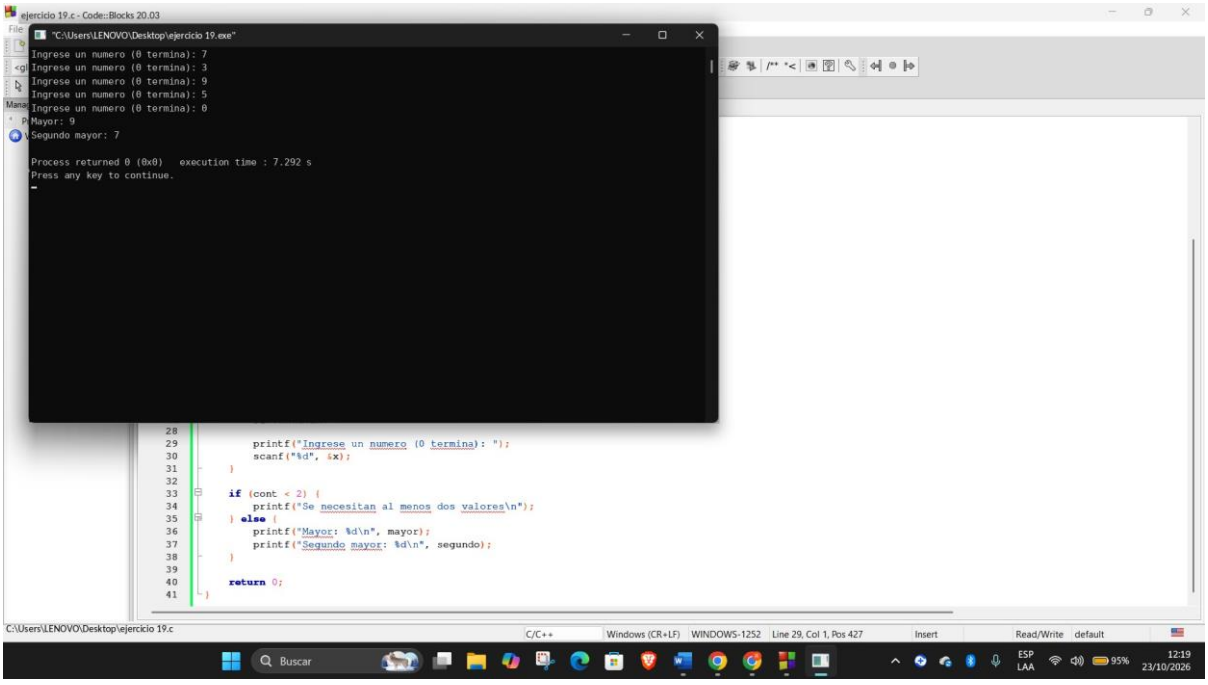
Decisión: El segundo mayor es 9. El enunciado pide informar si hay «menos de dos valores», es decir, cuenta valores ingresados, no distintos. Prueba: el primer 9 → mayor = 9; el segundo 9 no cumple x > mayor pero sí x > segundo (0) → segundo = 9; el 5 no cambia nada. Resultado: 9 y 9.

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)





Problema 1.20. Adivina el número

Desarrolla un programa que "piense" un número entre 1 y 100 (puede ser fijo para la práctica) y pida al usuario adivinarlo. Después de cada intento debe indicar "más alto", "más bajo" o "¡acertaste!" y al final mostrar cuántos intentos se usaron.

1. Análisis del problema

Idea general: El programa guarda un número fijo (37). El usuario intenta una y otra vez y en cada intento recibe una pista. Se cuenta cuántos intentos hace hasta acertar.

Entradas: los intentos del usuario (enteros), uno por vuelta.

Proceso: Se repite: leer el intento, sumar 1 al contador y compararlo con el secreto (si es menor, «Más alto»; si es mayor, «Más bajo»; si es igual, «¡Acertaste!»). El ciclo termina cuando intento = secreto.

Salida: Una pista después de cada intento y, al final, la cantidad de intentos usados

Restricción: El secreto está entre 1 y 100 (aquí es fijo, 37). Siempre se hace al menos un intento, por eso se usa un ciclo Repetir ... Hasta que, que revisa la condición al final

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Número secreto	secreto = 37	cte	entero
Dato de entrada (intento usuario)	intento	var	entero
Cantidad de intentos	cont	var	entero

3. Seudocódigo

```
Inicio
  secreto ← 37
  cont ← 0
  Repetir
    Leer intento
    cont ← cont + 1
    Si intento < secreto entonces
      Escribir "Más alto"
    Sino
      Si intento > secreto entonces
        Escribir "Más bajo"
      Si no
        Escribir "¡Acertaste!"
      Fin Si
    Fin Si
  Hasta que intento = secreto
  Escribir "Intentos usados: ", cont
Fin
```

4. Prueba de escritorio

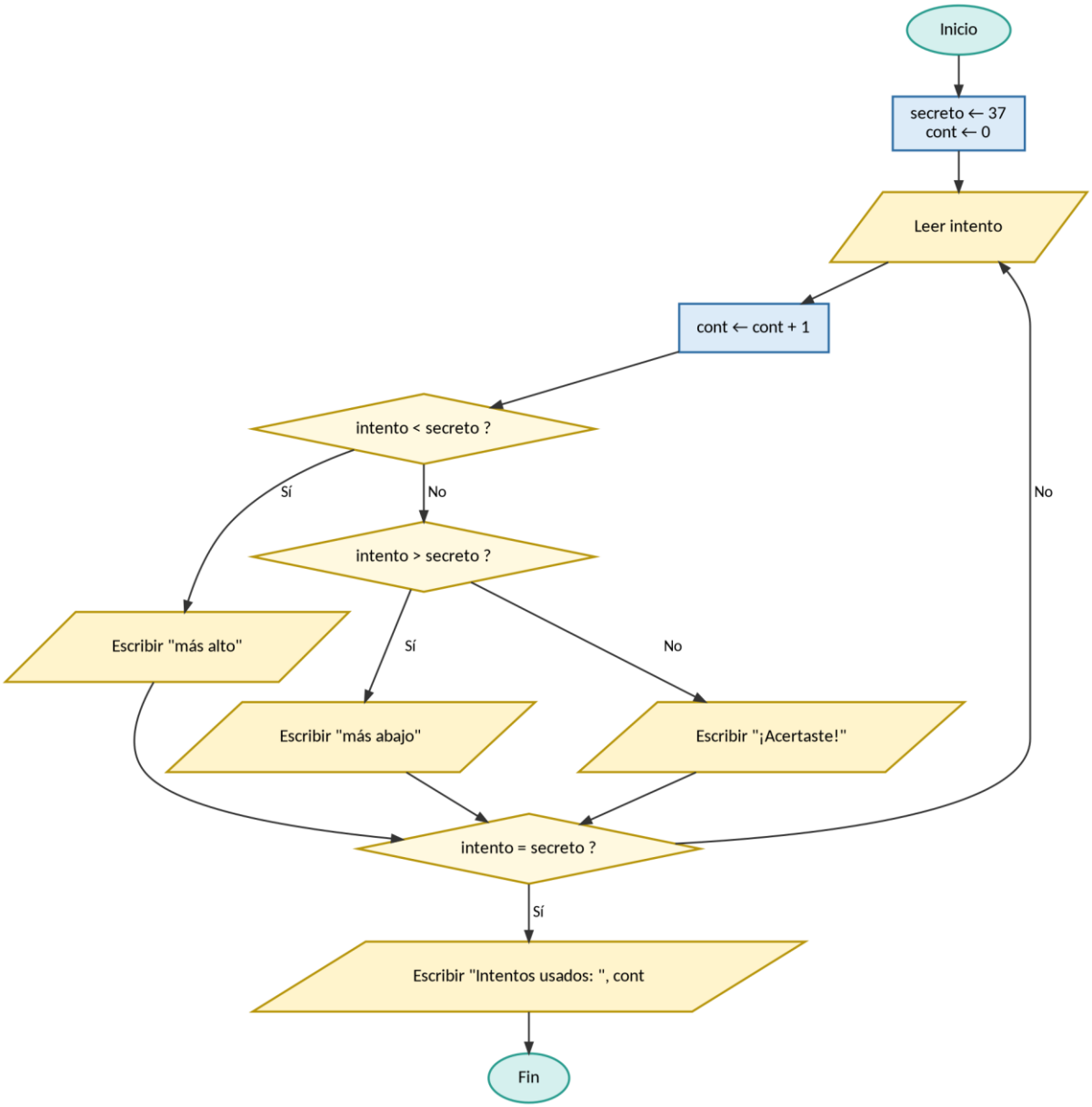
Caso	Intento (en orden)	Pistas	Salida final
Acierta a la primera	37	Acertaste	Intentos usados 1
Normal	50, 25, 37	Más bajo, Alto,	Intentos usados 2
Normal (más intentos)	10, 90, 40, 35, 37	Más alto, bajo, alto,	Intentos usados 2

Para pensar: Si jugaras con la mejor estrategia posible (dividir siempre el rango por la mitad), ¿cuál es el máximo de intentos que necesitarías para el rango 1-100? Justifica tu respuesta.

Serían 7 intentos: cada intento deja como máximo la mitad de candidatos (100 → 50 → 25 → 12 → 6 → 3 → 1) y el séptimo acierta; $2^6 = 64 < 100 \leq 128 = 2^7$.

Ejemplo con 37: 50, 25, 37 → 3 intentos, frente a hasta 100 probados en orden.

5. Diagrama de flujo



6. Código en C y ejecución (Code::Blocks)

